

Fostering Authentic Coding via a Web-Based Python IDE with Automated Structural Analytics

A Capstone

Presented to the Faculty of
College of Computing Studies
Pampanga State University

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Computer Science

by:

Del Mundo, Elijah Gabriel M.
Delos Reyes, Timothy Robin R.
Esguerra, Januard D.
Kabigting, Errol B.
Mendoza, Edward Arthur S.
Peralta, Francis Kenneth S.

TABLE OF CONTENTS**CHAPTER I: The Problem and its Background**

Introduction	1
Purpose and Description	3
Objectives of the Study	5
Significance of the Study	6
Scope and Delimitations	8
Conceptual Framework	9
Definition of Terms	11

CHAPTER II: RELATED LITERATURE AND STUDIES 13

The Impact of Generative AI on Programming Education	13
Superficial Learning and the Need for Guided Systems	14
Challenges in Maintaining Academic Integrity in Online Assessments	14
Data-Driven Approaches and Screen Monitoring for Cheating Detection	15
Asynchronous Task Queues for Scalable Systems	15
Secure Sandboxing in Online Code Execution	16
Automated Grading and Real-Time Analytics	16
Abstract Syntax Tree Model for Semantic Evaluation	17
Source Code Clone Detection Using Unsupervised Similarity Measures	17
Advancements in Plagiarism Detection for Python	18
Comparative Analysis of Selected Algorithms for Automated Evaluation	18
Software Quality Evaluation Using ISO/IEC 25010	20
Synthesis of the Reviewed Literature	20
SYSTEM TECHNICAL BACKGROUND	21
Front-End Development and User Interface	21
Back-End Framework and Database Management	22
Secure Execution and Asynchronous Task Queuing	22

CHAPTER III: Methodology 24

SOFTWARE DEVELOPMENT METHODOLOGY	25
REQUIREMENTS ANALYSIS AND DOCUMENTATION	30

List of Figures

Figure 3: System Work Plan	30
Figure 4: Flowchart Diagram	31
Figure 5: Use Case Diagram	32
Figure 6.1: Context Diagram (Level 0 Data Flow Diagram)	35
Figure 6.2: DFD Level 1	37
Figure 6.3: DFD Level 2 Process 1	39
Figure 6.4: DFD Level 2 Process 2	41
Figure 6.5: Level 2 Process 3	42
Figure 6.6: DFD Level 2 Process 4	43
Figure 6.7: DFD Level 2 Process 5	45
Figure 6.8: DFD Level 2 Process 6	47
Figure 7: Entity Relationship Diagram (ERD)	49
Figure 8: Gantt Chart	50
 IMPLEMENTATION PLAN	 52
Table 1: Implementation Plan	53
Table 2: Hardware and Software Requirements	54
 ISO/IEC 25010 SOFTWARE QUALITY MODEL	 56
 Respondents of the Study	 57
 RESEARCH TOOLS AND INSTRUMENTS	 59
Pre-Test and Coding Practices Assessment	59
Post-Test and System Evaluation Questionnaires	60
Data Gathering Procedure	60
Sampling Technique	61
 Statistical Treatment of the Data	 62
Data Analysis	64
References	64

CHAPTER I

INTRODUCTION

Under the Commission on Higher Education (CHED) Memorandum Order (CMO) No. 25 Series of 2015, Computer Science programs are mandated to produce graduates equipped with strong analytical, architectural, and problem-solving competencies. To cultivate these foundational skills, Python has emerged as one of the most dominant programming languages in both academic and industry settings (Hashmi, 2025). Its structural clarity and readability make it highly effective for teaching complex programming logic and computational thinking. However, the teaching of Python presents a distinct modern challenge. As Python has become the primary language associated with artificial intelligence development (Hashmi, 2025), it is also among the languages most accurately generated by AI-powered tools. Consequently, students learning Python are continuously exposed to accessible generative AI assistance. According to Garcia (2025), the increasing popularity of Generative AI has significantly influenced how students approach complex code creation, making them more inclined to bypass the manual coding process in favor of instantly generated solutions.

This immediate reliance on generative tools deprives students of the productive struggle necessary to develop authentic programming skills (Nasr et al., 2025). Although this issue affects learners across all educational levels, it becomes particularly critical during the third year of a Computer Science degree,

where academic demands increase significantly. In the core Programming Languages subject, students are expected to move beyond basic syntax and develop deeper competencies in algorithmic logic, object-oriented programming, and advanced data structures. However, when students depend heavily on generative AI to construct program logic or resolve structural errors, they risk bypassing the cognitive processes essential for mastering these advanced concepts. As a result, they fail to cultivate the architectural planning and analytical thinking skills required by the curriculum, reducing programming from an active problem-solving discipline into a largely passive activity.

Because students can easily conceal this lack of hands-on engagement by submitting polished AI-generated code, educators urgently require tools capable of monitoring the actual coding process. However, most existing educational platforms primarily evaluate the final output of a program, providing educators with limited insight into how students arrived at their solutions (Park et al., 2025). This creates a significant gap in monitoring student behavior during independent and unmonitored coding sessions. Recent studies highlight the importance of evaluating how students construct their code by focusing on their behavior, workflow, and progression rather than solely assessing the final result (Gao et al., 2025). In this context, tracking coding behaviors such as tab-switching and application shifting becomes particularly valuable. Frequent context-switching may serve as a behavioral indicator of cognitive offloading, where students repeatedly leave the coding environment to depend on external sources or

generated solutions instead of engaging in independent problem-solving.

To address this lack of visibility and reduce excessive reliance on generative AI, there is a critical need for a platform that goes beyond output-checking. To accurately assess genuine learning, the proposed platform operates on the principle that the evaluation of a student's programming skills (measured through AST structural logic analysis) combined with their programming behavior (measured through tab-switching and activity logs) directly reflects their true level of advancement. This holistic tracking allows instructors to formulate objective scoring rubrics based on authentic logic formulation rather than relying solely on the final printed code.

Therefore, this study introduces the Web-Based Python Integrated Development Environment (IDE) with Behavioral Tracking, AST Feedback, and Real-Time Analytics. To further promote academic integrity alongside the structural checking of AST, the system utilizes the Jaccard Similarity Algorithm to detect copied code among students (Adegbite & Kotzé, 2025). The platform is specifically tailored for third-year Computer Science students at Pampanga State University. Because these students are currently taking advanced courses like Programming Languages, they are expected to formulate complex algorithms independently without AI assistance, making them the ideal target users for this rigorous structural and behavioral evaluation.

PURPOSE AND DESCRIPTION

The general purpose of this study is to promote academic integrity and support the development of advanced programming logic among third-year computer science students at Pampanga State University through a Web-Based Python Integrated Development Environment (IDE). To achieve this, the web application incorporates the following core attributes:

Behavioral Tracking. The system includes a module that actively records observable student programming activities. It monitors code editing frequency, execution attempts, error occurrences, and tab-switching events. This tracking is implemented to analyze coding patterns and prevent negative cognitive offloading, ensuring that students engage in independent problem-solving during coding sessions rather than relying on external AI-generated solutions.

AST-Driven Feedback. An Abstract Syntax Tree (AST) code analysis module detects the presence and correct structural use of required programming constructs. To encourage productive struggle, the system provides automated structural feedback only post-submission, ensuring students attempt to formulate solutions independently first. The backend evaluates the structural logic to generate guidance when solutions possess missing elements, unnecessary complexity, or inefficient algorithms.

Real-Time Instructor Dashboard. A live dashboard gives educators a clear view of student performance. It allows teachers to track class progress, evaluate learning behaviors in a timely way, and identify students who need early

intervention. It translates the tracked programming skills (AST evaluation) and programming behavior into actionable data, allowing instructors to determine the student's true level of advancement instead of relying solely on final printed outputs.

Anti-Code Duplication. To promote academic integrity alongside structural checking, the platform utilizes the Jaccard Similarity algorithm. This mechanism measures the similarity of tokenized Python source code submissions to accurately detect copied code among students.

Controlled Interface. The IDE environment encourages manual coding by restricting specific interface actions. This includes limiting copy-paste functionalities during independent practice sessions and unmonitored assignments to enforce genuine coding effort.

STATEMENT OF THE PROBLEM

This study aims to design, develop, and evaluate a Web-Based Python Integrated Development Environment (IDE) equipped with behavioral tracking, AST-driven feedback, and a real-time instructor dashboard.

Specifically, this research seeks to answer the following questions:

1. What system features are necessary to address the challenges in evaluating the Python programming activities of third-year Computer Science students at Pampanga State University in terms of:
 - 1.1. Monitoring independent coding behavior and reliance on external

assistance;

1.2. Assessing the structural logic and required programming constructs of the code; and

1.3. Detecting peer-to-peer code duplication?

2. How will the core functional modules of the proposed system be developed using the V-Model Software Development Life Cycle (SDLC) framework, specifically in terms of:

2.1. Behavioral Tracking;

2.2. AST-Driven Structural Feedback;

2.3. Real-Time Instructor Dashboard; and

2.4. Anti-Code Duplication Mechanism?

3. What is the level of software quality of the developed system, as evaluated by third-year Computer Science students and IT educators, based on the ISO/IEC 25010 standards in terms of:

3.1. Functional Suitability;

3.2 Usability;

3.3. Performance Efficiency;

3.4. Reliability; and

3.5. Security?

OBJECTIVES OF THE STUDY

This section presents the general and specific objectives that guide the development and evaluation of the proposed system. These objectives serve as the foundation for the study's system design, data gathering procedures, and software quality evaluation.

General Objective

The primary objective of this study is to develop a Web-Based Python Integrated Development Environment (IDE) equipped with behavioral tracking, AST-driven structural feedback, and a real-time instructor dashboard to assist IT educators in monitoring student programming behaviors and promoting academic integrity.

Specific Objectives

Specifically, the study aims:

1. To design the system features necessary to address the challenges in evaluating the Python programming activities of third-year Computer Science students at Pampanga State University, specifically targeting the monitoring of independent coding behavior, the assessment of structural logic, and the detection of code duplication.
2. To develop the core functional modules of the proposed system using the V-Model Software Development Life Cycle (SDLC) framework, with the following specific developmental targets:
 - 2.1. To integrate a behavioral tracking module that monitors

tab-switching and limits copy-paste actions;

2.2. To implement a post-submission AST-driven structural feedback engine equipped with an objective scoring rubric to evaluate the student's true level of advancement;

2.3. To construct a real-time instructor dashboard for live monitoring of student progress; and

2.4. To deploy an anti-code duplication mechanism utilizing the Jaccard Similarity Algorithm.

3. To evaluate the level of software quality of the developed system, as assessed by third-year Computer Science students and IT educators, based on the ISO/IEC 25010 standards in terms of:

4.1. Functional Suitability;

4.2. Usability;

4.3. Performance Efficiency;

4.4. Reliability; and

4.5. Security.

SIGNIFICANCE OF THE STUDY

This study aims to provide a structured and monitored programming environment. The integration of behavioral tracking and automated structural feedback will benefit the following groups:

Third-Year Computer Science Students

The system will primarily benefit third-year programmers by providing an environment that encourages manual coding and promotes academic integrity. Through the Abstract Syntax Tree (AST) feedback, students will receive automated structural guidance. This supports their productive struggle, allowing them to practice genuine algorithmic thinking and independent problem-solving without immediate reliance on generative AI.

CCS Instructors

The platform will serve as an effective monitoring and formative evaluation tool for educators. Using the real-time dashboard, teachers will gain visibility into the actual coding habits and problem-solving workflows of their students. This allows them to quickly identify learners who require early intervention and easily spot academic dishonesty, such as code duplication, rather than relying solely on the final printed output.

Pampanga State University (PAMSU)

The successful implementation of this system will benefit the institution, specifically the College of Computing Studies (CCS), by providing a secure web-based educational tool. It supports the university's goal of delivering quality computing education by ensuring that students in higher-level programming courses cultivate verified and independent coding skills.

Other Computing Institutions and Independent Learners

Because the system is deployed as a web-based platform, its accessibility extends beyond the primary testing grounds. IT educators and programming

students from other institutions can utilize the IDE to conduct monitored coding sessions, access AST-driven structural feedback, and maintain academic integrity in their own respective programming environments.

Future Researchers

This study will serve as a valuable reference for future researchers who wish to explore behavioral tracking, AST-driven feedback, and software quality evaluation in educational platforms. The methods and results of this project, particularly the ISO/IEC 25010 software evaluation, can provide baseline data for other studies aimed at enhancing structural analysis and academic integrity in programming education.

SCOPE AND LIMITATION

The scope of this study encompasses the development and evaluation of a Web-Based Python Integrated Development Environment (IDE) tailored for the College of Computing Studies (CCS) at Pampanga State University (PAMSU). The system integrates a code editor, behavioral tracking, Abstract Syntax Tree (AST) code analysis, and a real-time instructor dashboard. The platform is designed to support various academic delivery modes, including in-class laboratory activities, take-home assignments, and independent practice sessions. The research focuses on the system's technical quality and functional effectiveness in promoting academic integrity and structural programming discipline among third-year Computer Science students, with data collection facilitated through ISO/IEC 25010 software quality evaluation.

Conversely, the study adheres to specific limitations and delimitations to ensure project feasibility. The evaluation respondents are restricted to third-year Computer Science students and CCS instructors at Pampanga State University. The IDE is strictly engineered for Python and does not support other programming languages. Technically, the system is designed for single-file Python programs and operates within a sandboxed environment with strict limits on execution time and memory usage. Behavioral tracking is delimited to in-browser actions, specifically tab-switching and copy-paste events; it explicitly excludes invasive monitoring tools such as webcam surveillance, microphone recording, screen capture, or AI-based text detection to prioritize data privacy and system accuracy. The Jaccard Similarity algorithm is used for code duplication flagging but does not determine the original authorship of the code. Furthermore, the system cannot prevent students from using secondary physical devices, such as smartphones, to seek external solutions. Finally, the platform requires a stable internet connection and does not provide offline functionality.

CONCEPTUAL FRAMEWORK

The conceptual framework of the study illustrates the systematic flow of resources, developmental procedures, and expected outcomes of the Web-Based Python IDE with Behavioral Tracking and AST-Driven Feedback. Utilizing an Input-Process-Output (IPO) model, this framework serves as a roadmap for designing, developing, and evaluating a structured educational programming environment. Ultimately, it guides the creation of a highly usable and reliable system, ensuring that the final software product meets the ISO/IEC 25010 software quality standards.

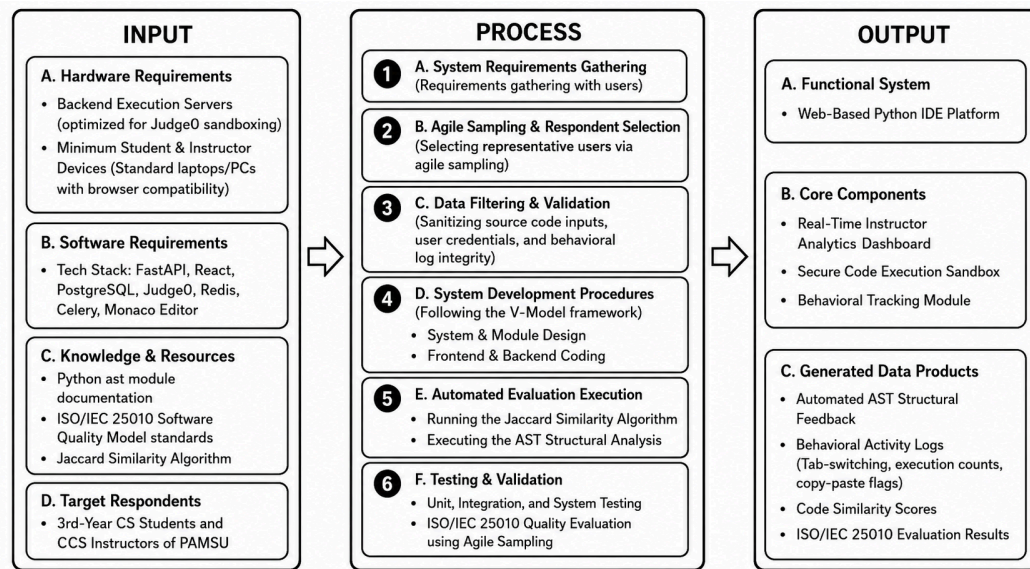


Figure 1: *Input Process Output*

The input phase establishes the foundational requirements needed for the study. This includes the specific hardware specifications, emphasizing backend execution servers optimized for Judge0 sandboxing alongside the minimum device requirements for students and instructors. It also outlines the necessary

software technologies to build the application, specifically the integration of FastAPI, React, PostgreSQL, Judge0, Redis, Celery, and Monaco Editor. Furthermore, it incorporates essential knowledge resources, primarily the Python ast module documentation, the Jaccard Similarity algorithm, and the ISO/IEC 25010 Software Quality Model standards. Lastly, it identifies the target respondents, namely the third-year Computer Science students and IT educators of the College of Computing Studies (CCS) at Pampanga State University (PAMSU).

The processing phase details the systematic methods used to develop, evaluate, and validate the system. It begins with requirements gathering and data validation to ensure the integrity of source code inputs, user credentials, and behavioral logs. System development strictly follows the V-Model Software Development Life Cycle (SDLC) framework, encompassing module design and both frontend and backend coding. This phase also executes the automated mechanisms, specifically the Jaccard Similarity algorithm and the AST structural analysis. The development concludes with rigorous testing stages (Unit, Integration, and System Testing) followed by the software quality evaluation based on the ISO/IEC 25010 model. To select the respondents for this evaluation, an agile sampling method is utilized.

The output phase showcases the final deliverables and the concrete data generated by the research. The primary outcome is the fully functional Web-Based Python IDE platform, featuring core components such as the real-time

instructor analytics dashboard, a secure code execution sandbox, and a behavioral tracking module. Additionally, the system yields specific generated data products, including AST structural feedback, detailed behavioral activity logs (such as tab-switching, copy-paste attempts, and execution counts), and code similarity scores, alongside the final system documentation and ISO/IEC 25010 quality evaluation results.

Through this Input-Process-Output framework, the study maps the transformation of system requirements into a validated educational tool. It equips instructors with actionable, data-driven tools to accurately monitor student progress, evaluate learning behaviors, and track coding activities in real time. For the third-year students, the platform enforces manual problem-solving, analyzes structural logic, and utilizes a configurable delay timer for feedback delivery to encourage productive struggle and authentic independent coding practices.

DEFINITION OF TERMS

The researchers operationally defined the following terms based on how they were used in the system and the study.

Abstract Syntax Tree (AST). In this study, it refers to the analysis module used by the system to automatically check the structural correctness of a student's Python code.

Behavioral Tracking. This refers to the system's capability to monitor student activities, such as tab-switching and code execution attempts, to ensure independent problem-solving and promote academic integrity.

Generative Artificial Intelligence (Gen AI). This refers to external automated tools, such as ChatGPT, that students might misuse to bypass the manual coding process.

Integrated Development Environment (IDE). This refers to the proposed web-based platform where third-year students write, execute, and practice their Python programming assignments.

ISO/IEC 25010 Software Quality Model. This refers to the internationally recognized standard used in this study to evaluate the system's Functional Suitability, Usability, Performance Efficiency, Reliability, and Security.

Jaccard Similarity Algorithm. This refers to the specific computational method used by the system to measure code similarity and detect copied Python submissions among students.

Real-Time Instructor Dashboard. This refers to the live interface that allows CCS instructors to actively monitor their students' coding behaviors, progress, and errors.

Sandboxed Environment. This refers to the secure, isolated execution area within the platform that runs student code under strict time and memory limits.

CHAPTER II

RELATED LITERATURE AND STUDIES

The Emergence of Python and its Connection to Artificial Intelligence

Python has emerged as one of the most dominant programming languages in both academic and industry settings (Hashmi, 2025). Its structural clarity and readability make it highly effective for teaching complex programming logic and computational thinking. However, the teaching of Python presents a distinct modern challenge. As Python has become the primary language associated with artificial intelligence development, it serves as the foundation for modern AI systems, including large language models and Generative AI. Its extensive ecosystem of specialized data science libraries makes it the primary language for machine learning. This deep integration with AI technologies has created a paradox in programming education. Because Generative AI tools are exceptionally proficient at writing Python code, students learning this specific language are highly susceptible to overreliance on AI-generated solutions.

Recent empirical evidence further validates this concern. A 2025 study on Python programming education by Li, Liu, and Dong revealed that while GenAI tools can expedite short-term task completion, excessive reliance leads to a phenomenon known as negative "cognitive offloading." In unregulated environments, students often develop a dependency on merely copying AI-generated code rather than

engaging in independent problem-solving, which severely limits long-term knowledge transfer and critical thinking. These findings strongly reinforce the necessity for specialized Python coding platforms like the Python Web IDE. By integrating behavioral tracking and post-submission AST evaluation, the platform is specifically designed to counteract passive cognitive outsourcing, limit external AI interventions, and enforce authentic algorithm development.

Generative AI in Programming Education

The rapid use of generative artificial intelligence tools such as ChatGPT has significantly changed programming education, particularly among Computer Science and Information Technology students. Nathaniel (2025) explored how students interact with AI-assisted coding tools and found that although these systems provide instant feedback and faster task completion, beginners easily develop dependency patterns. Similarly, Stoyanova et al. (2025) observed that AI tools alter student learning behaviors by shifting learners away from traditional problem-solving and independent debugging practices. While AI-assisted systems can help students complete coding tasks efficiently, they may bypass the deeper cognitive processes required for long-term programming mastery.

Recent studies further support concerns regarding overreliance on generative AI. Lepp and Kaimre (2025) identified a strong relationship between chatbot usage and student performance but emphasized that heavy AI dependence may weaken conceptual understanding. Chen et al. (2025) similarly reported that novice

programmers frequently rely on AI-generated solutions merely to complete assignments, resulting in temporary improvements in grades without corresponding gains in programming knowledge. Wu (2026), through a meta-analysis, confirmed that excessive dependence on AI tools contributes to reduced higher-order thinking and weak long-term skill retention. In addition, Elnaffar et al. (2025) found that a majority of recent studies associate AI overreliance with superficial learning behaviors. These findings collectively highlight the need for guided programming environments that encourage independent problem-solving, logical reasoning, and authentic coding practice rather than direct answer dependency.

Challenges in Programming Assessment and Evaluation

Programming assessment remains one of the most difficult aspects of programming education because instructors must evaluate not only the correctness of outputs but also the logical structure, originality, and problem-solving process behind student solutions. Recent studies emphasize that automated programming assessment systems are increasingly integrated into web-based IDE environments to improve grading efficiency, feedback quality, and programming instruction. Parvathy et al. (2025) explained that automated code assessment systems enhance programming education by evaluating not only output correctness but also code structure and originality, allowing instructors to provide faster and more consistent feedback.

Studies further suggest that modern automated evaluators are evolving beyond traditional grading functions into intelligent learning support systems. Research involving AI-assisted code review systems demonstrated that automated code-edit analysis can identify patterns in student debugging behavior and logic formulation, enabling instructors to better understand how learners construct programming solutions. These findings indicate that automated evaluators can support not only assessment efficiency but also deeper analysis of programming behavior and problem-solving processes.

Recent literature also highlights the growing integration of automated assessment systems into online programming platforms and IDE-like environments. Jung et al. (2026) noted that many automated feedback systems embedded in online coding environments still focus primarily on output correctness rather than deeper cognitive processes such as reasoning, structural logic, and long-term skill development. This limitation emphasizes the need for educational programming systems capable of analyzing both student outputs and programming behavior.

Similarly, Philippine-based studies indicate that novice programming students frequently experience difficulties in logic formulation, syntax construction, and algorithmic thinking, resulting in low confidence and poor performance in programming activities. Talandron-Felipe (2025) emphasized that many students struggle to translate theoretical programming concepts into practical coding solutions, particularly in environments where learning support and guided feedback mechanisms are limited. Studies on programming anxiety among

computer studies students further revealed that stress, low self-confidence, and difficulty understanding coding structures negatively affect student engagement and performance during programming tasks (Yu et al., 2025).

The literature further suggests that traditional assessment methods in programming courses often focus primarily on final outputs rather than the actual coding process used by students. As a result, instructors may experience difficulty identifying weaknesses in student logic formulation, debugging behavior, and structural understanding. These challenges support the need for more interactive and supportive programming environments that provide automated structural feedback, guided evaluation, and activity monitoring features. In this context, the development of a web-based Python IDE with automated structural analytics and behavioral tracking mechanisms may help improve student understanding, engagement, and programming performance in local computing education settings.

Academic Integrity and Behavioral Tracking

The transition to online learning environments created major challenges in maintaining academic integrity during programming assessments. Without direct classroom supervision, opportunities for cheating and unauthorized collaboration increased significantly. Socrates et al. (2022) explained that traditional manual monitoring methods are insufficient in online examination settings and

emphasized the need for technology-assisted monitoring systems to support secure assessments.

To address these concerns, recent studies have introduced behavioral tracking and screen-monitoring mechanisms in online coding platforms. Alkinani (2026) developed an AI-supported cheating detection system capable of analyzing audio, video, and user activities in real time. However, simpler monitoring mechanisms remain widely adopted because of their practicality and lower computational requirements. Socrates et al. (2022) designed a web-based examination platform capable of monitoring user screen behavior and detecting browser tab-switching attempts. Their system successfully warned and restricted students who violated examination policies, demonstrating that tab-switch monitoring can effectively support academic integrity in online assessments.

Behavioral tracking has also become increasingly important in programming education. Behavioral tracking refers to the monitoring and recording of user interactions such as tab-switching, code execution attempts, keyboard activity, idle time, and submission patterns during programming activities. Bhatlawande et al. (2025) developed CodeNest, a web-based coding platform that monitors restricted behaviors such as copy-paste attempts and tab-switching during timed coding activities. Similarly, Pokhrel et al. (2025) integrated AI-based proctoring and activity tracking into an online examination platform to improve assessment security. Khairnar et al. (2025) further proposed ProctorCode, a framework that tracks application switching and inactive behavior during practical programming

examinations. These studies demonstrate that behavioral tracking systems are increasingly used to promote academic honesty, strengthen assessment reliability, and encourage independent logic formulation in online programming environments.

Integrated Development Environments (IDEs)

Integrated Development Environments (IDEs) provide centralized programming workspaces that combine tools such as source code editors, debugging utilities, compilers, interpreters, syntax highlighting, and code execution features. These functionalities simplify software development by allowing programmers to write, test, and debug applications within a single environment. According to Helbing et al. (2025), IDEs improve development efficiency, reduce coding errors, and enhance user productivity in both professional and educational settings.

In programming education, IDEs provide structured and user-friendly environments that allow students to focus more on algorithm development and problem-solving rather than manual configuration tasks. Real-time debugging and code feedback features help learners identify programming mistakes more efficiently and improve their understanding of coding structures and syntax.

Advancements in cloud computing and browser technologies have also contributed to the development of web-based IDEs. Unlike traditional desktop IDEs, web-based IDEs allow users to access programming environments directly through web browsers without requiring local installation. These systems provide advantages such as cross-platform accessibility, centralized deployment, automatic updates, and easier classroom management. As online learning environments continue to expand, web-based IDEs have become increasingly valuable in educational institutions because they support flexible and accessible programming instruction.

Automated Grading and Real-Time Analytics

Automated grading systems have become increasingly important in programming education because they reduce instructor workload and improve evaluation consistency. Bato and Pomperada (2025) developed an automated grading system with integrated student performance analytics and found that automated assessment tools provide real-time insights that help instructors identify struggling students early. Ramizares and Ramizares (2025) similarly reported that automated grading systems reduce human error and improve consistency in academic evaluation.

The integration of analytics into automated assessment systems also allows instructors to monitor submission patterns, evaluate student progress, and provide timely interventions. These capabilities support more efficient classroom

management while minimizing grading fatigue among educators. Because of these advantages, automated grading platforms are increasingly adopted in programming courses and online coding environments.

Abstract Syntax Tree (AST) Analysis

An Abstract Syntax Tree (AST) is a tree-based representation of the logical structure of source code. Instead of evaluating the exact textual appearance of a program, AST organizes code into structural components such as variables, loops, functions, and conditional statements. This allows systems to analyze the semantic meaning and organizational structure of code more accurately than traditional text-based comparison methods.

AST has become widely used in compilers, code analyzers, plagiarism detection systems, and automated grading platforms because it evaluates programming logic rather than superficial formatting differences. Nurhasan et al. (2025) developed an AST-based grading model for Python and found that structural analysis reduced grading inaccuracies caused by different coding styles. Oliveira et al. (2025) also introduced PyDesignWizard, a tool that uses AST analysis to evaluate programming design practices and algorithm structure among beginner programmers.

Compared with traditional string-matching techniques such as Regular Expressions (Regex), AST-based evaluation is more resilient because it ignores formatting differences such as spacing, blank lines, and variable naming

variations. Parvathy et al. (2025) demonstrated that AST-based grading systems improved grading efficiency and produced more meaningful feedback compared with conventional text-matching approaches. These findings support the use of AST as an effective approach for evaluating programming logic and semantic correctness in educational coding systems.

Source Code Similarity and Plagiarism Detection

Detecting source code similarity is important in programming education because students may intentionally modify variable names or formatting to bypass plagiarism detection systems. Martinez-Gil (2024) explained that Jaccard Similarity remains an effective technique for code similarity analysis because it provides accurate results while requiring relatively low computational resources.

Recent studies have shown that combining structural and token-based analysis improves plagiarism detection accuracy. Senjaya et al. (2025) reported that plagiarism detection becomes more difficult when students apply small structural modifications to copied code. To address this issue, Gandhi et al. (2024) developed a Python plagiarism detection system that combined text-based and structural analysis techniques to generate a unified similarity score with high accuracy.

Although advanced approaches such as TF-IDF and Cosine Similarity are effective for textual document analysis, they can become computationally expensive when applied to programming syntax. In contrast, Jaccard Similarity

operates using set-overlap comparison, making it faster and more suitable for real-time code similarity analysis. By comparing syntax tokens extracted from source code, Jaccard Similarity provides a lightweight yet effective approach for identifying structural code duplication in online programming environments.

Secure Sandboxing in Online Code Execution

Sandboxing is a security technique used to safely execute untrusted or potentially harmful code inside isolated environments. In online programming platforms, sandboxing prevents submitted programs from accessing sensitive system resources or damaging the main server infrastructure. Chen and Wang (2025) explained that sandboxed execution environments isolate potentially harmful code from the host system, reducing security risks during automated code execution.

Modern online coding systems commonly implement Docker-based sandboxing to strengthen security and improve scalability. Johnson (2025) developed a coding platform that utilized Judge0 as its primary execution engine and demonstrated that Docker-based isolation effectively protected the system from malicious code execution. Similarly, Putra et al. (2025) explained that temporary Docker containers provide isolated environments that are automatically removed after execution, minimizing security vulnerabilities. Rabin et al. (2025) further emphasized that containerized testing environments improve both system safety and execution reliability in automated grading platforms.

Research on the “Learn to Code Fast” platform (2026) also showed that integrating Judge0 with modern web technologies enabled secure real-time code execution without requiring heavy local server configurations. These findings establish sandboxing as a critical component of modern web-based IDEs and automated programming assessment systems because it ensures safe, stable, and scalable code execution.

Asynchronous Task Queues for Scalable Systems

Online programming platforms must handle multiple code submissions simultaneously while maintaining system stability and responsiveness. The Real Python Research Group (2026) explained that asynchronous background task processing is necessary for systems that manage heavy workloads such as automated code evaluation. Instead of processing every task directly within the main application, asynchronous systems distribute workloads into separate background queues.

Technologies such as Redis and Celery are commonly used to implement asynchronous task queues in modern web systems. Redis functions as a message broker that temporarily stores pending tasks, while Celery processes these tasks asynchronously in the background. This architecture prevents system slowdowns and allows platforms to remain responsive even when many students submit programming exercises simultaneously. As a result, asynchronous task processing has become an important component of scalable online coding environments.

Software Quality Evaluation Using ISO/IEC 25010

ISO/IEC 25010 is an international software quality evaluation framework used to assess whether software systems meet user and organizational requirements. PacificCert Quality Assurance (2025) described the framework as a comprehensive standard for evaluating software quality characteristics such as functional suitability, usability, reliability, security, and performance efficiency.

Recent studies have applied ISO/IEC 25010 in evaluating educational and web-based systems. Alifya et al. (2025) found that functional suitability and usability strongly influence user satisfaction in academic platforms. Suryani and Pratama (2025) successfully applied the framework in evaluating an e-learning platform, while Centeno et al. (2025) emphasized that usability and learnability are especially important for beginner-oriented programming tools.

Because this framework evaluates both technical performance and user experience, ISO/IEC 25010 is widely used in assessing educational technologies and software systems. In this study, the framework serves as the primary basis for evaluating the overall quality, usability, reliability, performance, and security of the proposed Python Web IDE platform.

Synthesis of the Reviewed Literature

The reviewed literature collectively supports the development of a web-based programming platform that integrates automated assessment, behavioral

monitoring, secure execution, and semantic code evaluation. Studies on generative AI revealed that excessive reliance on AI-generated programming solutions may weaken independent problem-solving and logical reasoning skills among beginners, highlighting the need for guided educational systems that encourage authentic programming practice.

The literature further established that programming instructors and beginner students continue to face challenges in logic formulation, structural understanding, and effective assessment. Existing programming evaluation methods often focus only on final outputs while neglecting the actual coding process and problem-solving behavior of students. These limitations emphasize the importance of educational systems capable of evaluating both program correctness and programming logic.

Studies on academic integrity demonstrated that behavioral tracking mechanisms such as tab-switch monitoring, activity logging, and restricted behavior detection are effective in supporting secure online programming assessments. In addition, studies involving automated grading and AST-based semantic evaluation established that structural code analysis provides more accurate and meaningful feedback than traditional text-based grading methods. Research on plagiarism detection further showed that combining structural analysis with lightweight similarity measures such as Jaccard Similarity improves the reliability of automated code comparison systems.

Moreover, studies involving Docker-based sandboxing, Judge0 integration, Redis, and Celery confirmed the importance of secure and scalable infrastructures for handling simultaneous code execution and automated grading tasks. Finally, the ISO/IEC 25010 framework provides a comprehensive basis for evaluating the proposed system in terms of functionality, usability, performance efficiency, reliability, and security.

Collectively, the reviewed studies establish the theoretical and technical foundation for Python Web IDE as a web-based educational programming environment that uniquely integrates post-submission AST-based semantic evaluation and behavioral tracking. This specific approach allows for independent logic formulation before system intervention, supported by plagiarism detection, secure sandboxed execution, and scalable automated assessment specifically designed for programming instruction and classroom evaluation.

SYSTEM TECHNICAL BACKGROUND

This section provides the technical background for the web-based Python coding platform. It discusses the different technologies, frameworks, and the overall system structure. This information gives a clear view of the technology stack that supports the core functions of the system, especially the post-submission AST structural feedback and behavioral tracking features.

Front-End Development and User Interface

For the graphical user interface, the system is built using React, a popular JavaScript library for creating interactive websites. Standard HTML5 is used to structure the web content, such as navigation bars and forms. To make the design clean and modern, the developers use Tailwind CSS. This framework allows the team to easily manage the layout and styling of the pages. JavaScript handles the dynamic parts of the site to ensure it responds quickly to user actions. Since the platform needs a built-in code editor, the Monaco Editor is integrated. This is the exact same code engine that powers professional development environments, giving the students a reliable and comfortable workspace to formulate their logic and write their Python scripts. To access the platform, users only need a standard web browser.

Back-End Framework and Database Management

For data management, the system uses PostgreSQL. It is a highly reliable database system that securely stores user accounts, teacher modules, coding exercises, and automated system logs, such as the AST pass/fail results and tab-switching counts. To connect the user interface with this database, FastAPI is utilized. FastAPI is a modern and fast web framework for building APIs in Python. It handles the core logic of the system, specifically the Abstract Syntax Tree (AST) analysis and the Jaccard Similarity calculations. These components are critical because they allow the system to automatically check, compare, and provide post-submission structural feedback on the submitted Python scripts.

Secure Execution and Asynchronous Task Queuing

Compiling and testing code can consume significant server resources. To safely execute the submitted scripts, the platform integrates Judge0. It is an open-source sandboxing environment that isolates the code execution. This means that if a student accidentally runs an infinite loop or writes potentially malicious code, the main server remains secure and operational. Additionally, because multiple students might submit their laboratory assignments and mock quizzes at the exact same time, the system must manage heavy network traffic efficiently. To solve this, asynchronous queuing is set up using Redis and Celery. Redis acts as a message broker that temporarily holds pending AST evaluation tasks, while Celery takes those tasks and processes them asynchronously in the background. This entire setup ensures that the coding platform remains stable and highly responsive even when an entire class utilizes it concurrently.

CHAPTER III

METHODOLOGY

This chapter presents the methods and procedures that the researchers will use to design, develop, and evaluate the Web-Based Python Integrated Development Environment (IDE). It outlines the research design, the software development life cycle, the target respondents, and the data gathering instruments. The methodology ensures that the platform functions effectively for third-year Computer Science students and IT educators at the College of Computing Studies (CCS) of Pampanga State University (PAMSU) while maintaining high software quality standards. The study utilizes a combined Developmental and Descriptive-Quantitative research design. Developmental research focuses on the systematic design, creation, and technical evaluation of technological products. Alongside this, the descriptive-quantitative approach is applied to gather numerical data that accurately measures the software quality and overall functional effectiveness of the newly developed system.

This combined research method is highly suitable for the objectives of the study. The developmental phase covers the actual programming and integration of the system's core features following the V-Model Software Development Life Cycle (SDLC). This involves building the Abstract Syntax Tree (AST) code analysis, setting up the behavioral tracking module, deploying the anti-code duplication mechanism, and creating the real-time instructor dashboard. On the other hand, the descriptive-quantitative phase is executed during the system

evaluation and data collection stages. Utilizing an agile sampling method, the researchers will distribute survey instruments based on the ISO/IEC 25010 Software Quality Model to the target respondents. This approach strictly evaluates the technical standards of the system in terms of Functional Suitability, Usability, Performance Efficiency, Reliability, and Security, focusing on software validation rather than experimental learning measurements.

SOFTWARE DEVELOPMENT METHODOLOGY

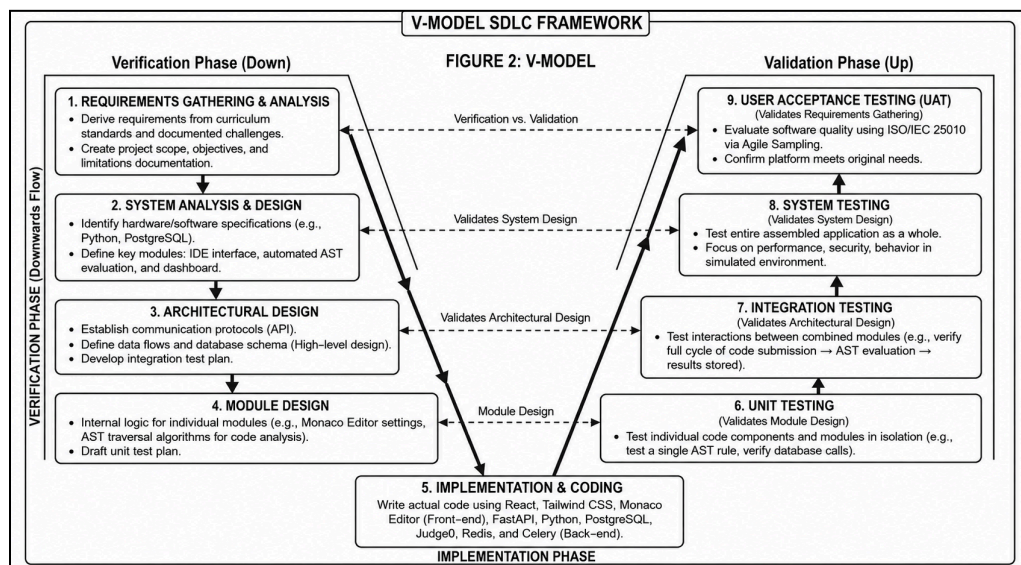


Figure 2: V-Model

As illustrated in Figure 2 below, the researchers will use the V-Model Software Development Life Cycle (SDLC) framework to develop the Web-Based Python IDE system. Developed by Barry Boehm in 1979, the V-Model is divided into three main phases. The left-hand side represents the verification phase, consisting of sequential levels where requirements are gathered and the system design is conceptualized. The base represents the coding phase where the software

is actually programmed. The right-hand side represents the validation phase, where the system is systematically tested against the designs to ensure the final product satisfies the functional requirements and user needs.

Requirements Gathering and Analysis. In this stage, the researchers will dedicate time to thoroughly establishing the functional and non-functional requirements of the project. Rather than conducting formal needs assessment interviews, the system requirements will be derived from existing computer science curriculum standards, documented academic challenges, and operational observations regarding programming logic evaluation. Additionally, a documentation plan will be constructed to define the project scope, core objectives, and delimitations.

System Analysis and Design. After defining the requirements, the researchers will identify the hardware and software specifications necessary for developing the application. This phase will outline the essential technologies and define the high-level architecture of the key modules, specifically the IDE interface, the automated AST evaluation engine, and the real-time instructor dashboard.

Architectural Design. In this phase, the researchers will establish the communication protocols, such as the REST API endpoints, and map the relationships between specific system components. Throughout this process, data flows and the PostgreSQL database schema will be drafted as a high-level design. Furthermore, an integration test plan will be developed to track data handoffs

between modules.

Module Design. In this phase, the researchers will create detailed, low-level designs for each specific component to ensure internal correctness. The internal logic for individual modules (such as the Monaco Editor constraints and the AST traversal algorithms for structural code analysis) will be strictly defined. A unit test plan will also be drafted to verify individual module functionality prior to integration.

Implementation and Coding. In this phase, the actual programming of the system will commence, utilizing the designs and requirements established in the previous phases. The front-end will be developed using React, Tailwind CSS, and the Monaco Editor, ensuring a responsive and controlled coding interface. Concurrently, the back-end will be constructed using FastAPI, Python, PostgreSQL, Judge0, Redis, and Celery to handle server-side operations, asynchronous processing, and secure sandboxed code execution.

Unit Testing. This phase will focus on testing the individual code components and modules in isolation to ensure all micro-functionalities operate correctly. Multiple isolated tests (such as testing a single AST validation rule or verifying discrete database queries) will be conducted by the developers to identify and resolve bugs at the lowest level.

Integration Testing. This phase will test the combined modules from the previous phase to verify data handoffs and system interactions. A series of tests will be conducted to ensure the integrated modules work together without errors,

specifically tracing the full cycle from code submission, through the Celery queue and AST evaluation, and finally confirming the results are properly stored in the database.

System Testing. This phase will involve testing the fully assembled application to evaluate its overall stability. The developers will focus on performance limits, execution security, and system behavior in a simulated deployment environment. This rigorous technical testing will continue until the platform is fully stabilized for end-user deployment.

User Acceptance Testing (UAT). The fully developed and stabilized Web-Based Python IDE will be evaluated by its intended users to validate if the initial requirements were successfully met. Utilizing an agile sampling method, third-year Computer Science students and IT educators from Pampanga State University will test the platform in real-world programming scenarios. This phase acts as the primary data collection stage of the study, wherein the respondents will evaluate the system using a survey instrument based on the ISO/IEC 25010 Software Quality Model, specifically measuring its Functional Suitability, Usability, Performance Efficiency, Reliability, and Security.

REQUIREMENTS ANALYSIS AND DOCUMENTATION

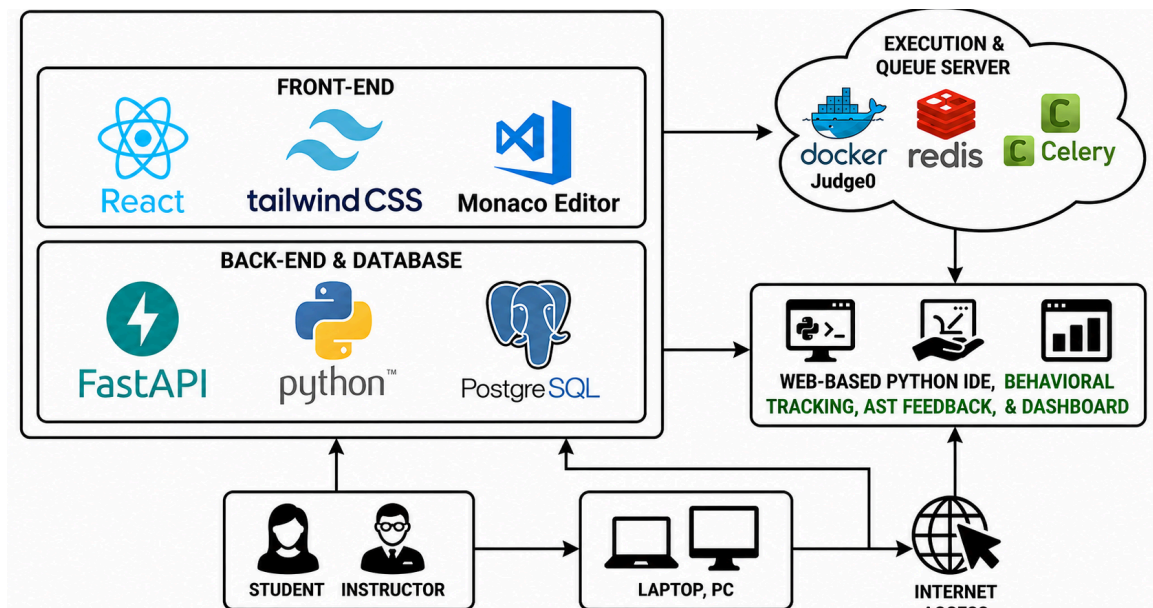


Figure 3: System Work Plan

Illustrated in Figure 3 above are the different languages and tools that will be utilized during the developmental phase of the system. The system will be designed to be user-friendly and will provide four core functions essential to its users: a Web-Based Python IDE, Behavioral Tracking, AST Feedback, and a real-time Dashboard. Initially, it will generate reports about student progress, behavioral logs (such as tab-switching), and code similarity scores. This will help the instructors efficiently monitor coding sessions and manage academic integrity easily. In addition, the instructors will be able to create and deploy coding tasks on the system by defining specific Abstract Syntax Tree (AST) rules which the students will view and solve on their end. They will also manage their respective classes by adding tasks and tracking structural programming progress.

Furthermore, individual students will have the ability to write, execute, and submit their Python code within the integrated web-based IDE, allowing them to receive automated structural feedback after a configurable delay.

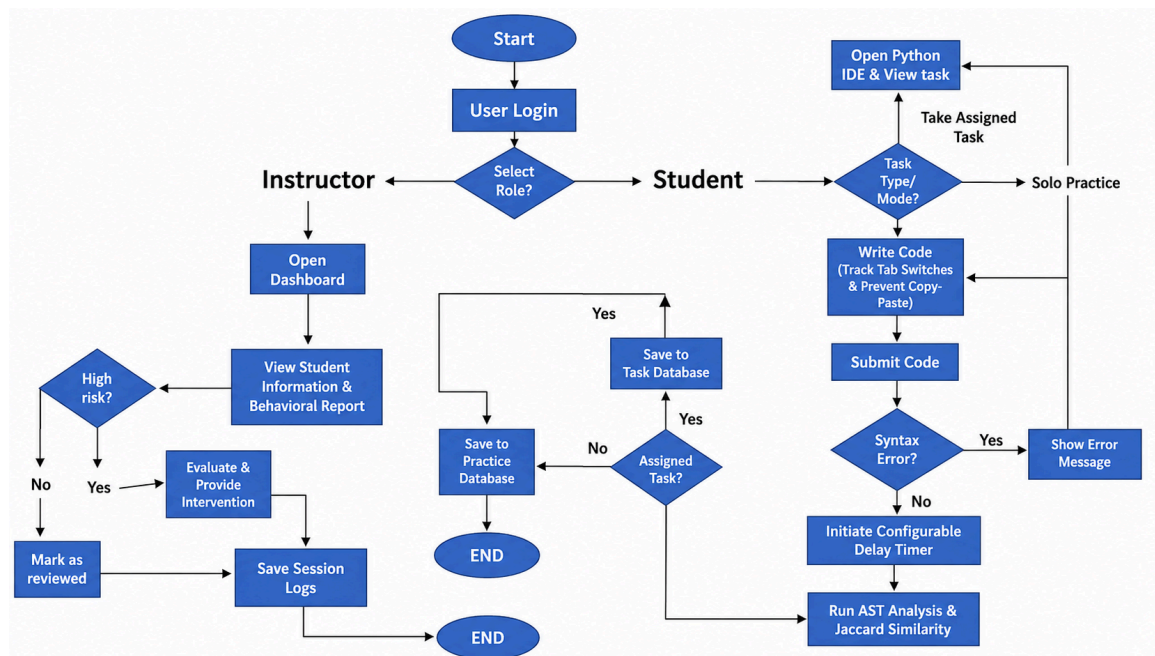


Figure 4: *Flowchart Diagram*

The diagram depicted in Figure 4 describes the logical flow of the proposed system. The process of accessing the features and functions of the system is demonstrated. The flowchart begins with the user login and role selection for both the students and the instructor, and both paths end upon completing their system access, coding tasks, independent practice sessions, and structural reviews. There are different types of shapes and symbols (such as rectangles, diamonds, oblongs, and arrows) that can be seen in the figure. To give a more in-depth idea of how the flowchart works, a short definition is provided for the following: The oblong is the Terminal Symbol, which describes the

beginning and ending of the flowchart; terms like Start and END are commonly used here. The diamond, also known as the Decision Symbol, acts as the one that decides how the flow should route, such as determining the user role, choosing between taking an assigned task or entering the solo practice module, checking for syntax errors, or evaluating high-risk behavior. The rectangle is known as the Process symbol. It represents the diagram's primary actions, such as writing code, initiating the configurable delay timer, running the AST analysis to provide automated structural feedback, tracking student progress on the dashboard, and saving the session logs. This is known as the most common component in a flowchart.

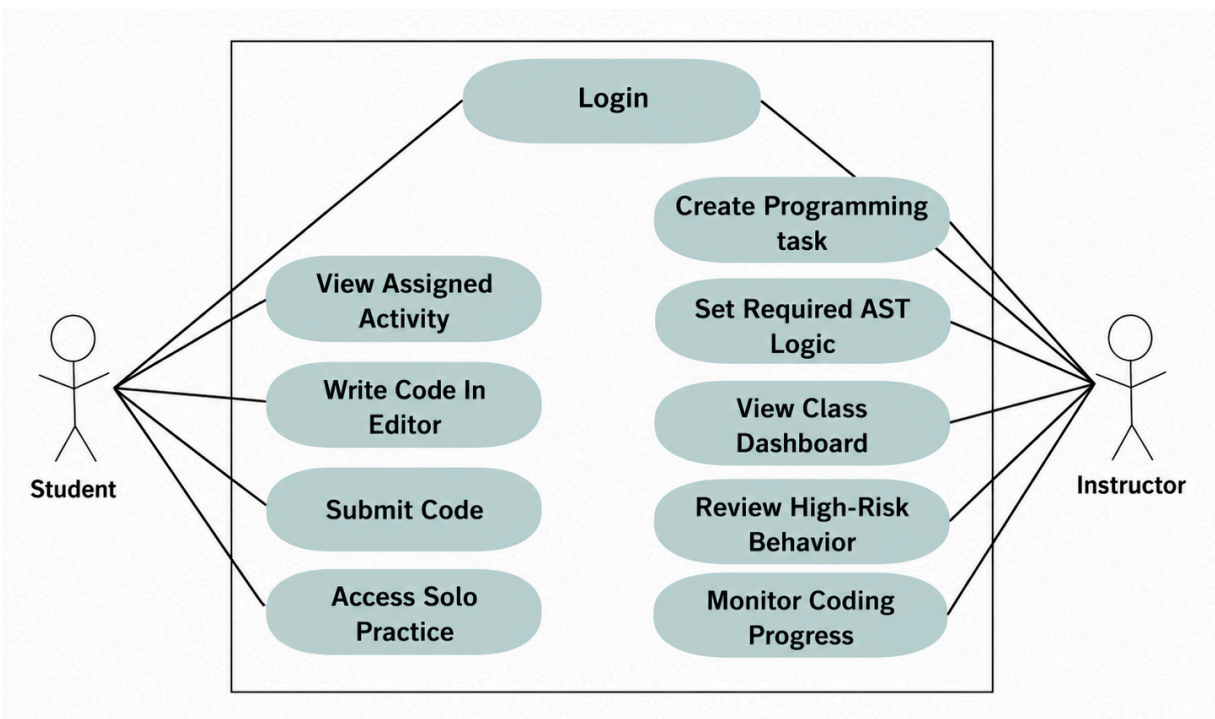


Figure 5: *Use Case Diagram*

Figure 5 illustrates the Use Case Diagram, presenting a visual representation of how the student and instructor will interact with the Web-Based Python IDE. The bounding box represents the scope of the system. The lines represent the associations between the users and the use cases. The stick figures act as the primary actors representing the student and instructor interacting with the system. The oval shapes are the use cases that describe the specific functions the student and instructor will perform.

The system includes a secure login feature for both actors. For the instructor, after logging in, there are five primary use cases: Create Programming Task, Set Required AST Logic, View Class Dashboard, Review High-Risk Behavior, and Monitor Coding Progress. There will be further actions connected to these use cases, allowing the instructor to define the structural rules for the coding tasks, monitor the overall class coding progress, review outputs from solo practice sessions, and check behavioral logs for any suspicious activities.

For the students, after logging in, they will be able to perform several use cases: View Assigned Activity, Write Code In Editor, Submit Code, and Access Solo Practice. They will be able to read their assigned programming problems, formulate their Python solutions directly within the platform, and submit their answers. Furthermore, the system provides a capability for students to practice independently using the built-in modules, where they will receive automated AST-driven structural feedback to guide their coding sessions.

Regarding data management, the instructor will be able to add, edit, and update the parameters for the programming tasks and AST rules, as well as retrieve analytics for the dashboard to monitor the continuous coding progress of the class. Students will be able to add and submit their Python code for both assigned activities and independent practice sessions. All the data entered (including the students' code submissions, solo practice logs, AST feedback scores, and behavioral reports) will be securely stored in the system database.

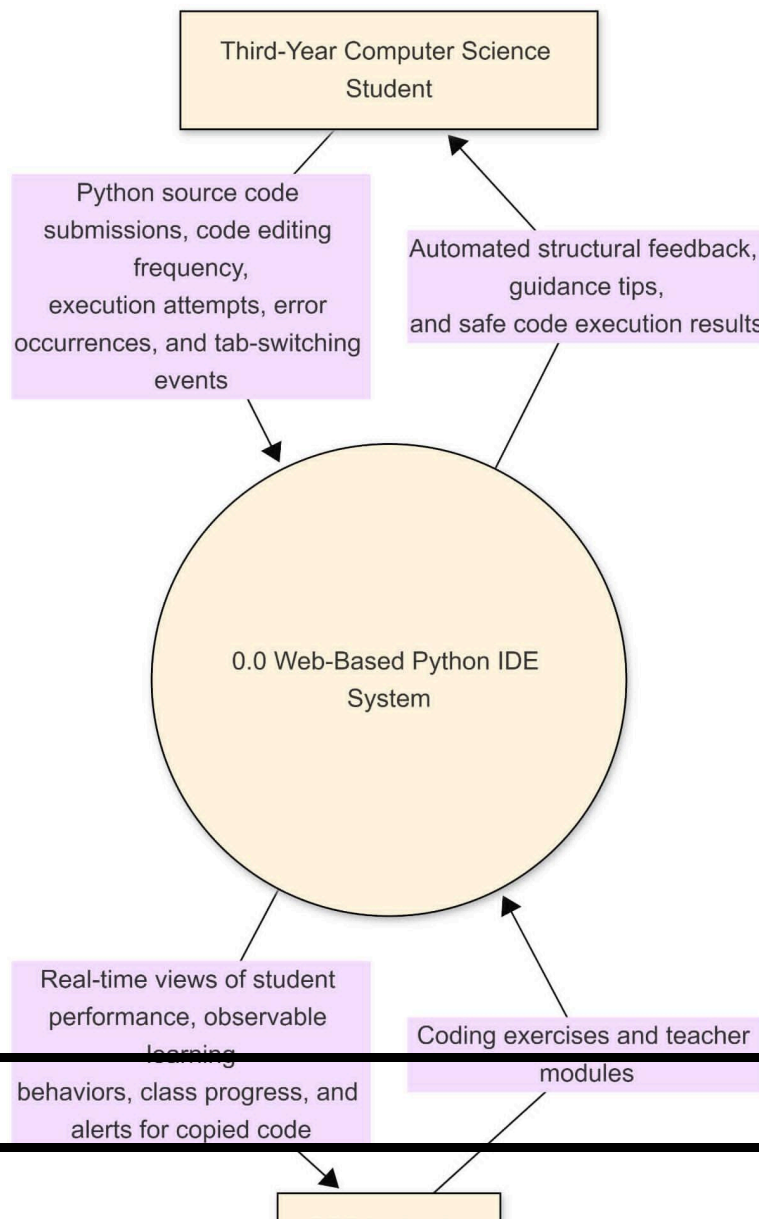


Figure 6.1: *Context Diagram (Level 0 Data Flow Diagram)*

The Context Diagram illustrates the comprehensive architecture of the Web-Based Python IDE System and its interactions with external entities. The central node represents the entire system, built using React and Tailwind CSS for the frontend, FastAPI for the backend, and PostgreSQL for database management. It interacts with two primary user groups:

- **Third-Year Computer Science Students:** Students access the platform via a web browser, inputting their Python source codes and interface actions. The system outputs isolated code execution results, automated structural feedback, and guidance tips.
- **CCS Instructors:** Educators input coding exercises and teacher modules into the system. The system provides them with a real-time dashboard displaying student performance, behavioral activity logs, class progress, and alerts for potentially copied code.

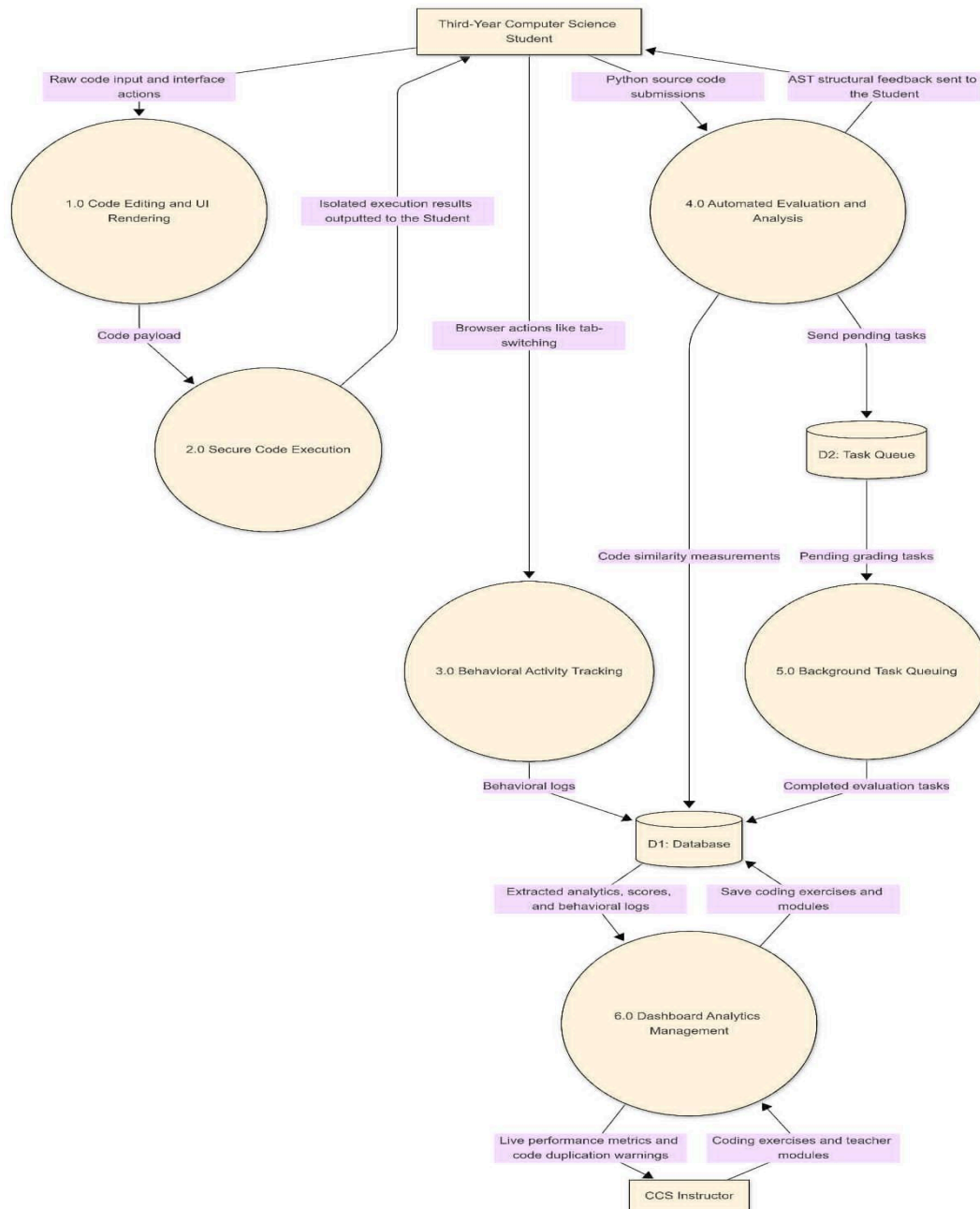


Figure 6.2: DFD Level 1

The Level 1 Data Flow Diagram breaks down the main system into its six core operational sub-processes and data stores.

- Process 1.0 & 2.0 (Interface & Execution): The system first handles user interactions and code editing through a React interface powered by the Monaco Editor. The submitted code payload is securely executed in a sandboxed environment using Docker and Judge0, isolating potentially harmful code from the main server.
- Process 3.0 & 4.0 (Tracking & Evaluation): Simultaneously, the FastAPI backend actively records observable programming behaviors, such as tab-switching. It also performs automated Abstract Syntax Tree (AST) structural analysis and calculates code similarity.
- Process 5.0 (Task Queuing): To maintain system stability during heavy class usage, a background task queuing mechanism utilizes Redis as a message broker and Celery to process grading tasks asynchronously.
- Process 6.0 & Data Stores: All processed data, including logs and scores, are stored securely in a PostgreSQL database. This data is consequently fetched to render real-time analytics on the instructor dashboard.

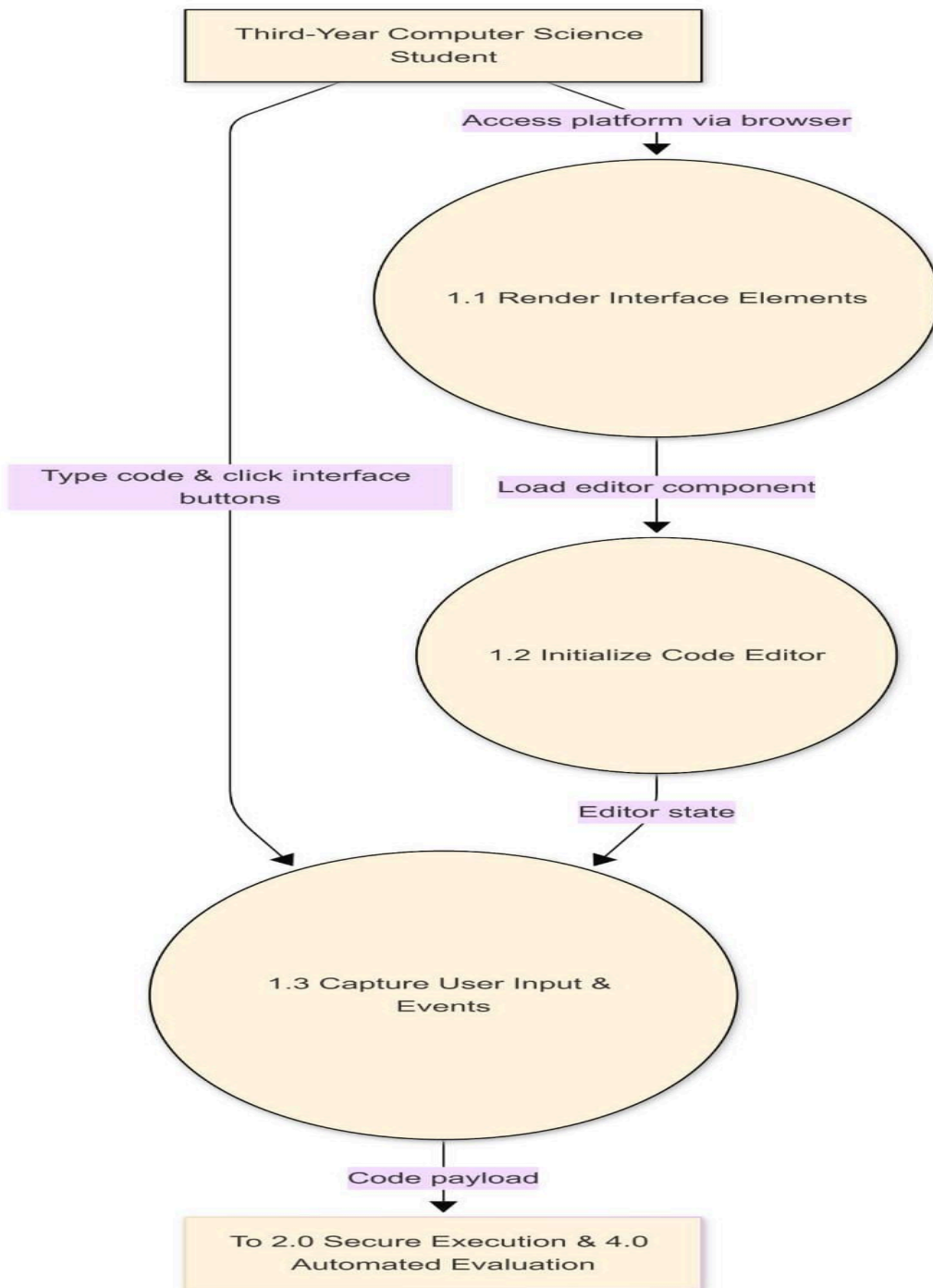


Figure 6.3: DFD Level 2 Process 1

This diagram details the physical implementation of the frontend interface.

The user interface is rendered using React and Tailwind CSS to establish a clean and modern layout.

Inside this interface is the Monaco Editor, the same engine utilized by Visual Studio Code, it is initialized to provide a professional coding environment. JavaScript captures dynamic user events, such as typing codes and clicking execution buttons, compiling them into a code payload that is sent to the backend for processing and execution.

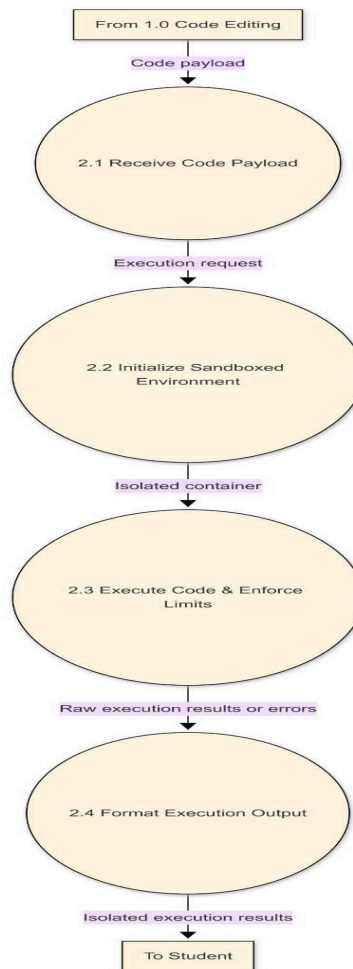
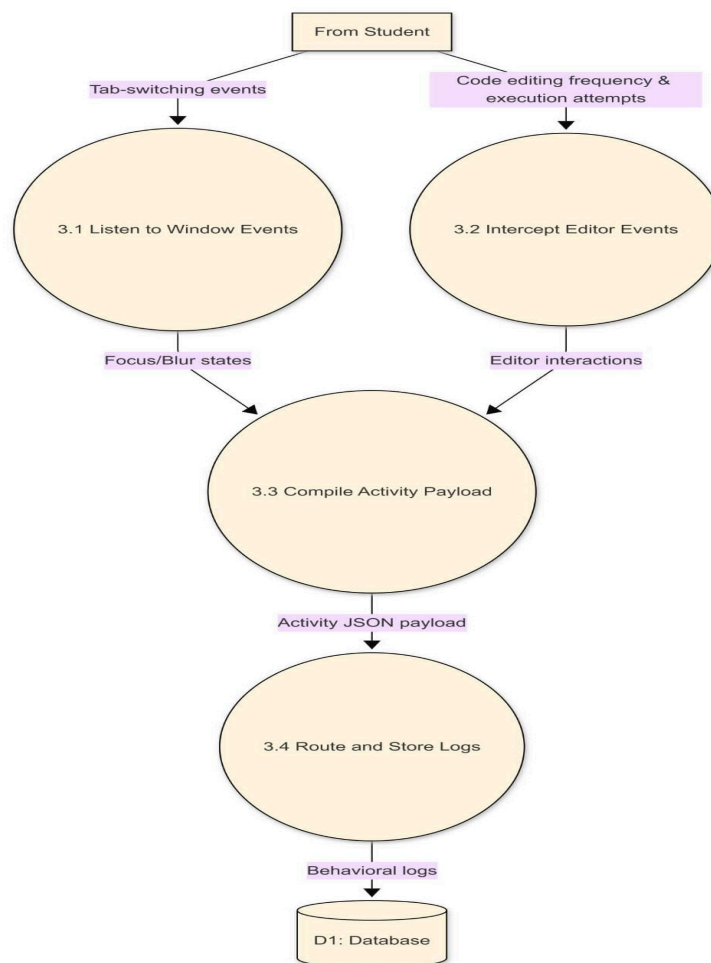


Figure 6.4: DFD Level 2 Process 2

This diagram illustrates the secure isolation of student code submissions. Upon receiving the code payload, the system spins up an isolated, sandboxed environment utilizing Docker and the Judge0 execution engine. This sandboxing mechanism enforces strict limits on execution time and memory usage. By separating the execution layer from the main server, the system ensures that infinite loops or malicious code cannot crash the platform, allowing it to safely



format and return the raw output back to the student's browser.

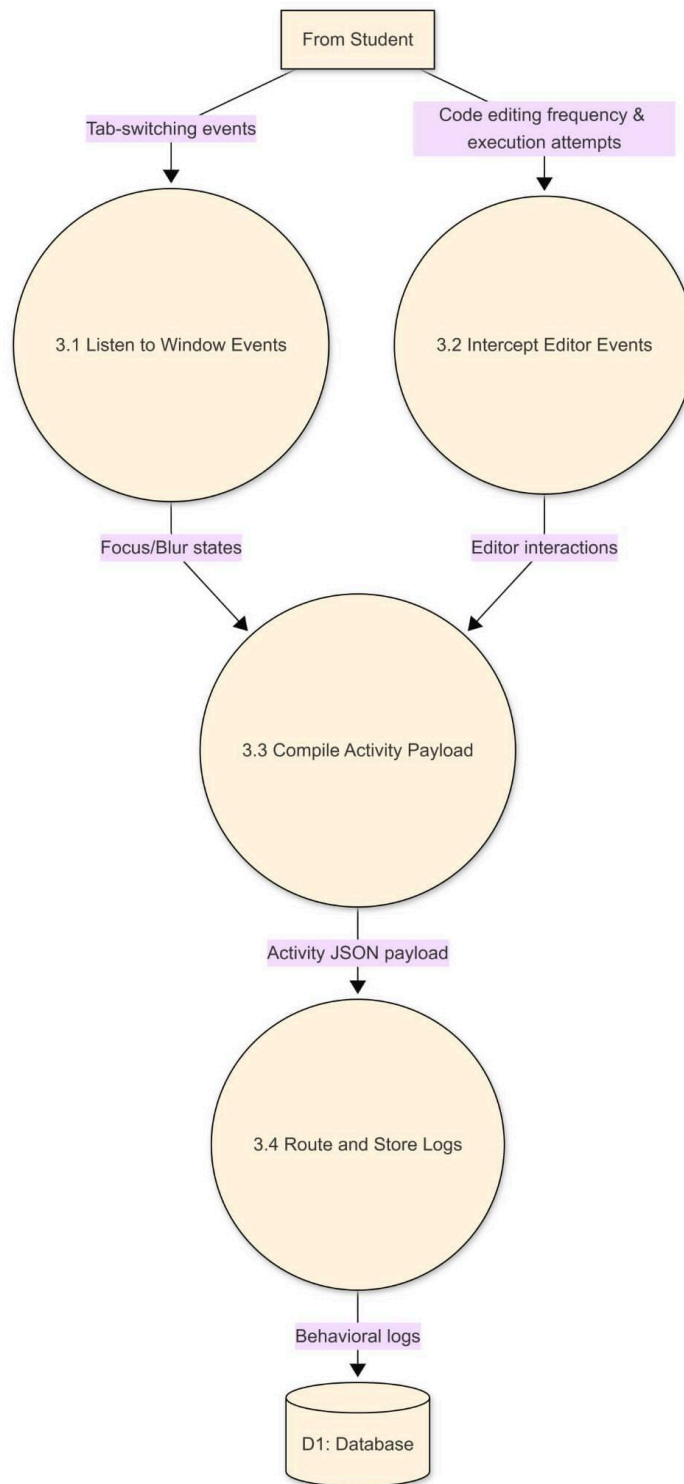


Figure 6.5: *Level 2 Process 3*

This diagram maps out the non-invasive behavioral tracking module designed to ensure independent problem-solving. The system actively monitors observable, in-browser activities rather than utilizing invasive tools like webcams or screen recordings. It intercepts browser window events to detect tab-switching and monitors editor activity to track code editing frequency, execution attempts, and error rates. These interactions are compiled into an activity payload, routed through the FastAPI backend, and stored as behavioral logs in the database for instructor review.

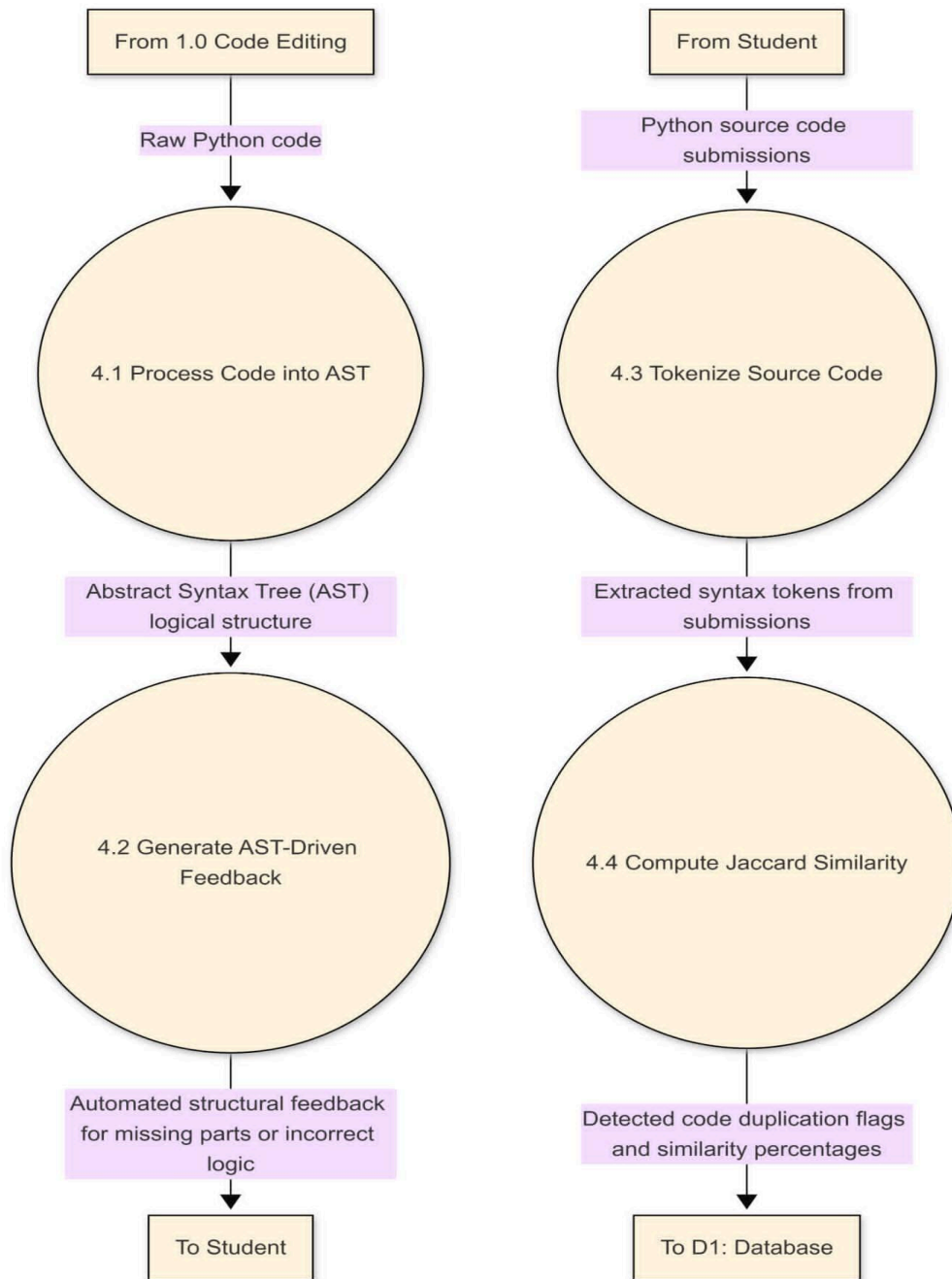


Figure 6.6: DFD Level 2 Process 4

This diagram breaks down the two primary backend grading mechanisms managed by FastAPI.

- AST Semantic Evaluation: First, the raw Python code is parsed into an Abstract Syntax Tree (AST) to evaluate its logical structure rather than just plain text. This AST is passed through a rule-based engine that checks for the correct implementation of programming constructs and automatically generates structural feedback.
- Anti-Plagiarism Mechanism: Concurrently, the source code is tokenized into syntax features. The system applies the Jaccard Similarity algorithm to measure the percentage of shared unique tokens across multiple student submissions. This lightweight computational method efficiently detects code clones and forwards the similarity percentages and duplication flags to the instructor dashboard.

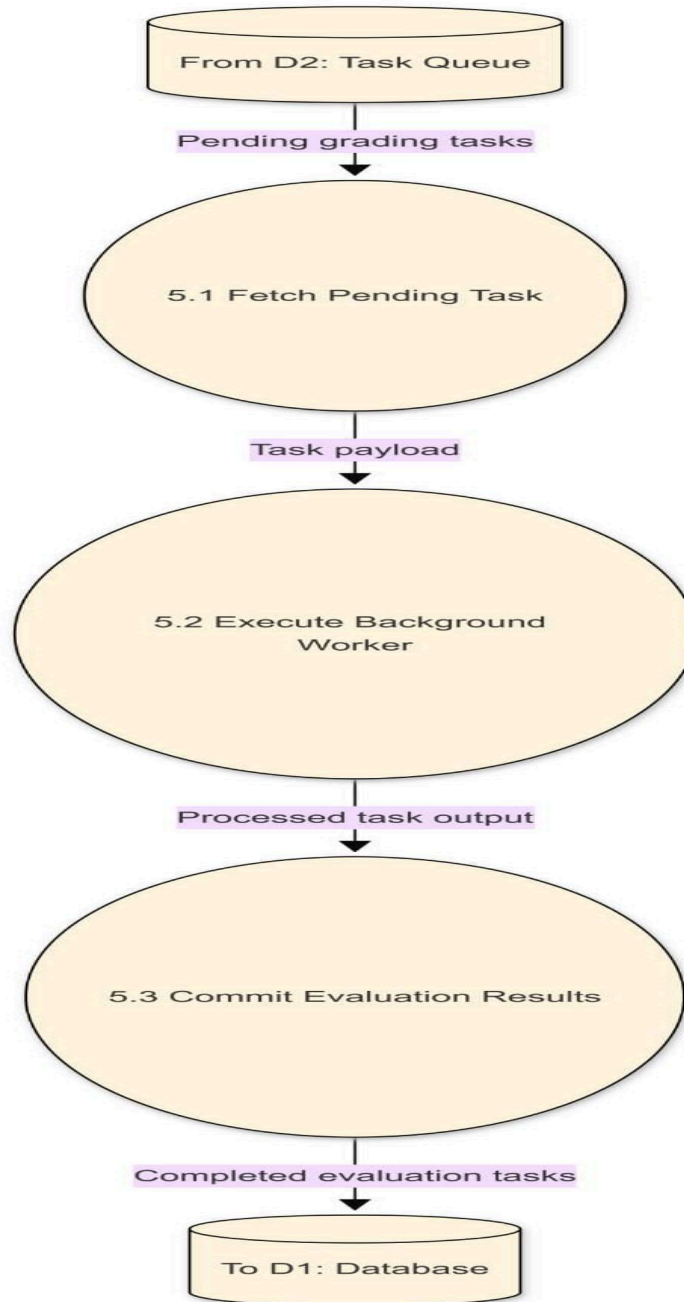
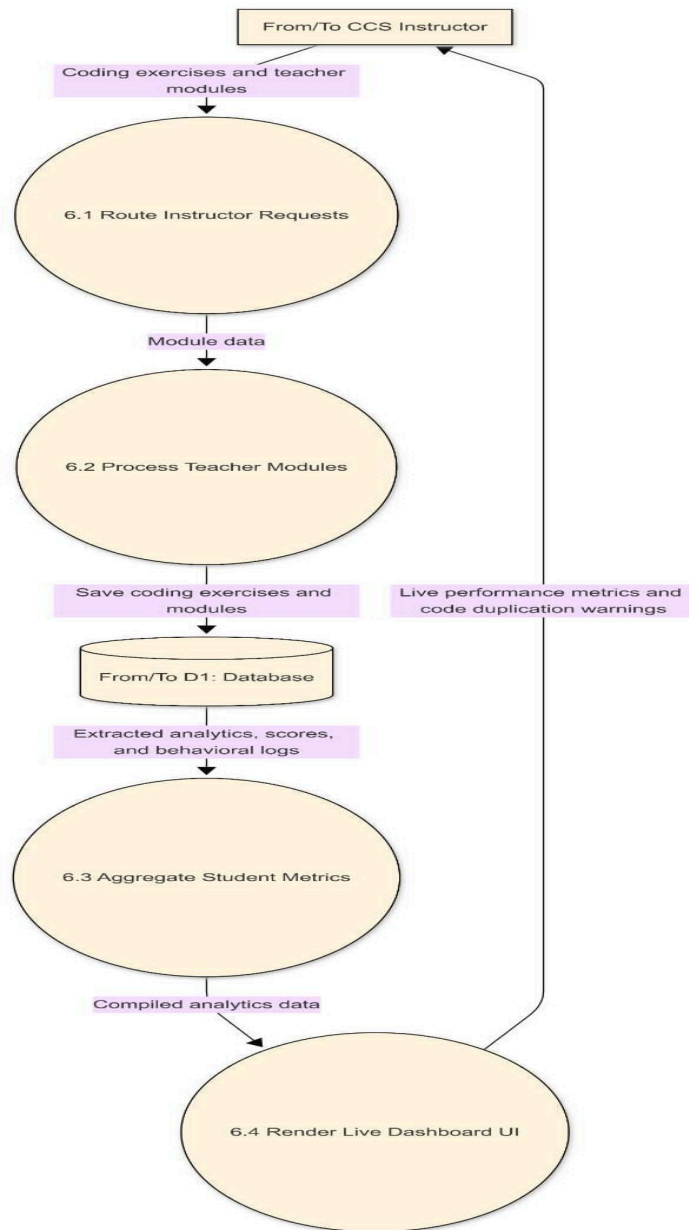


Figure 6.7: DFD Level 2 Process 5

This diagram illustrates the asynchronous workflow used to maintain system stability during high network traffic. The system continuously fetches pending grading tasks from the Task Queue. A background worker then executes

the heavy processing logic without interrupting the main web server.

Once the evaluation is finalized, the results are securely committed to the main Database, ensuring that multiple simultaneous submissions do not cause the platform to crash.



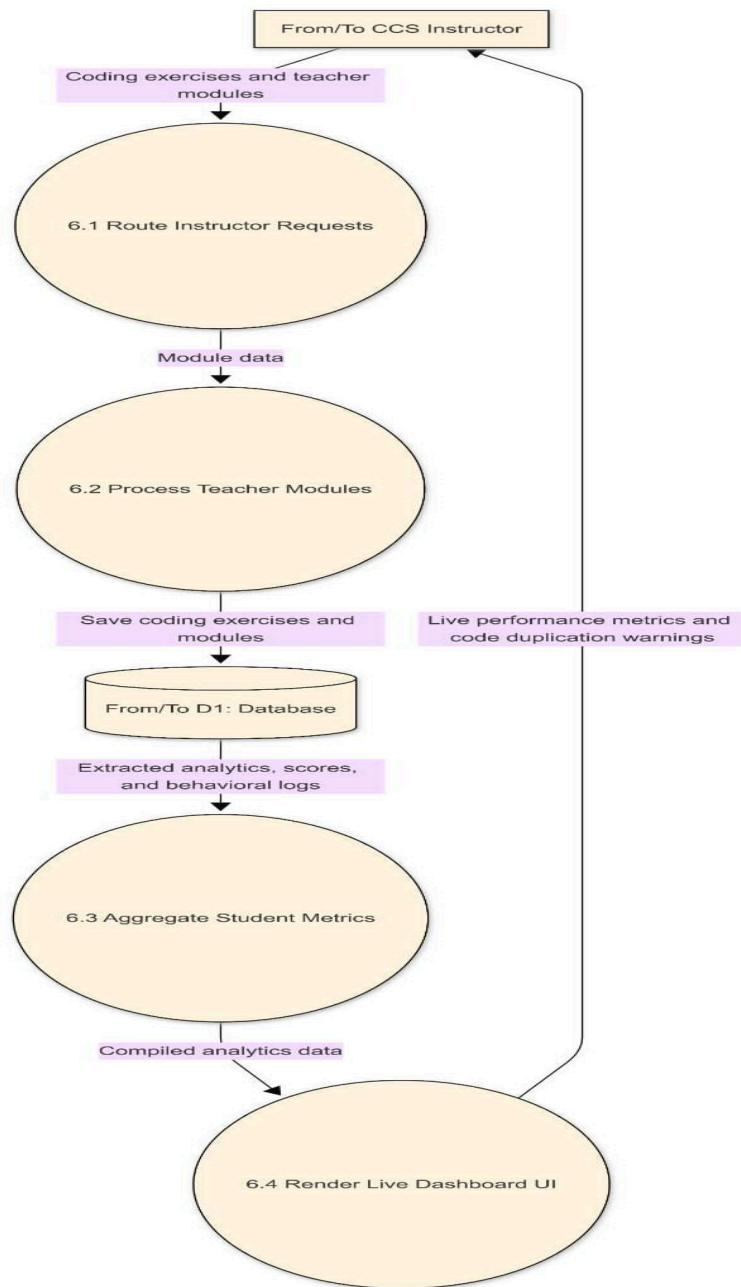


Figure 6.8: DFD Level 2 Process 6

This diagram details the data flow for the instructor's management interface. When an instructor uploads coding exercises and teacher modules, the

system routes and processes this data, saving it securely into the database. Concurrently, the system aggregates raw student data, including extracted scores, behavioral logs, and duplication flags from the database. This data is then compiled and rendered into the live dashboard interface, providing the instructor with real-time performance metrics and immediate warnings regarding academic integrity.

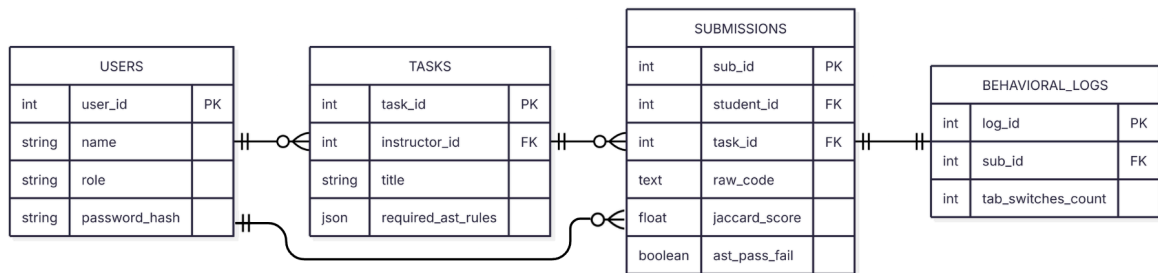


Figure 7: Entity Relationship Diagram (ERD)

The diagram illustrated in the figure depicts the relationships between the database entities, emphasizing the core information the system will handle. It shows that the 'SUBMISSIONS' entity has the most cardinalities connected, indicating its central importance in the system for processing AST-driven feedback and code similarity analysis. Similarly, the 'USERS' and 'TASKS' entities also have multiple connections; these entities demonstrate that role-based access and task assignment features are fundamental. Most of the entities possess one-to-many relationships with each other, aside from submissions to behavioral logs, which maintain a strictly one-to-one relationship. A user can manage or own one-to-many tasks and submissions, and a single task can accommodate

one-to-many submissions. On the other hand, a single submission links to exactly one corresponding behavioral log to securely record its specific tab-switching events.

SCHEDULE TIMELINE

The figure illustrates the time period from February to May that the researchers invested in the proposal phase of the project, starting with title conceptualization and culminating in the Proposal Oral Defense scheduled for May 20-22.

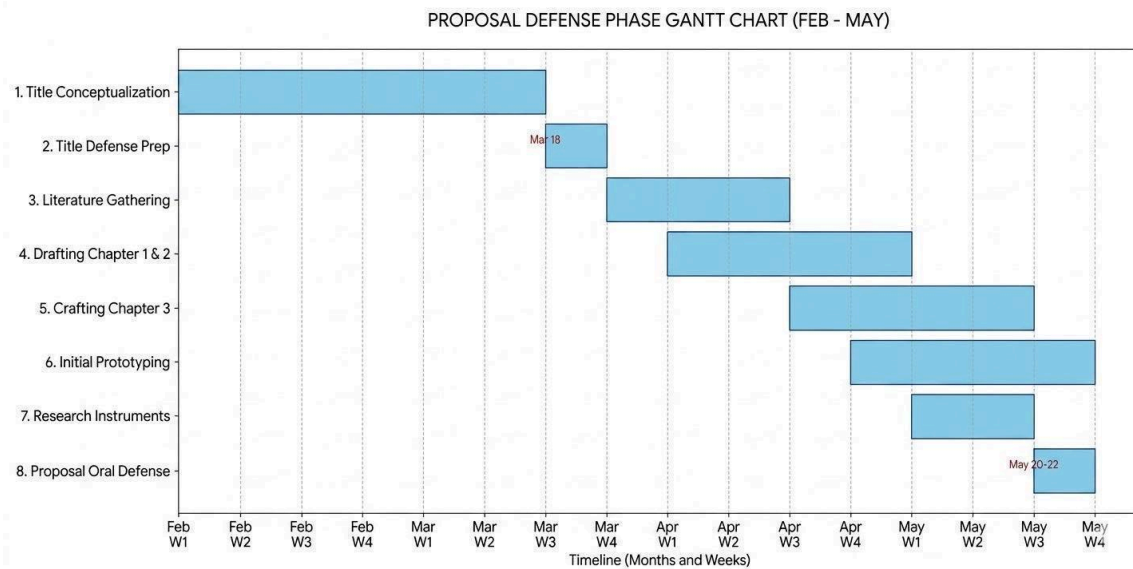


Figure 8: Gantt Chart

RESPONSIBILITIES

The development of the Web-Based Python IDE will require assigning specific roles and responsibilities to the researchers. The performance shown by the researchers in their respective roles will be crucial to the success of the system

development.

Project Manager. This role will be responsible for planning, managing, and assigning tasks to the team members. It will monitor and ensure that the tasks are delivered properly and on time throughout the development phase.

Frontend Designer. The designer will be responsible for formulating the content of the user interface based on the established system requirements. It will focus on creating a user-friendly but efficient interface for the students and instructors.

Database Designer. This role will be responsible for creating physical and logical database designs based on the interface and system requirements, specifically structured for securely storing AST rules, code submissions, and behavioral logs.

Programmer. This role will be responsible for ensuring the technical quality and functionality of the system prior to deployment. This includes conducting rigorous isolation tests on the secure execution sandbox and validating the software using the ISO/IEC 25010 quality model.

Software Tester. This role will be responsible for ensuring the quality and functionality of the system, particularly the secure execution sandbox.

Document Writer. The writer will be responsible for drafting, editing, and revising the manuscript, while collaborating with other team members to continuously align the document with the research instruments and system architecture.

IMPLEMENTATION PLAN

The proposed system is a web-based Python Integrated Development Environment (IDE) designed to facilitate the monitoring of coding sessions for third-year Computer Science students at the College of Computing Studies (CCS) of Pampanga State University (PAMSU). It allows CCS instructors to access real-time behavioral tracking, view automated structural feedback through Abstract Syntax Tree (AST) analysis, and ensure academic integrity during programming activities.

The main tasks for implementing this platform include system analysis and design, development and coding, testing, and user deployment. Analysis and design involve defining the AST rules, designing the user interface, and structuring the database. Development and coding focus on building the front-end, setting up the back-end API, and integrating the sandbox environment for secure code execution. Testing ensures that the Jaccard Similarity algorithm, behavioral tracking, and overall system performance work accurately. Finally, user deployment includes orienting the students and instructors on how to navigate the platform.

The resources required to implement the system include hardware, software, the researchers, and the target users. Hardware requirements consist of standard laptops or personal computers with internet access. Software needs include code editors, web browsers, cloud servers, and the necessary frameworks like React, FastAPI, and PostgreSQL. The researchers will serve as the main

development team. The intended users are the CCS instructors and the third-year Computer Science students.

After the initial development, the system will be deployed to the selected CCS instructors and students for pilot testing and technical evaluation utilizing the ISO/IEC 25010 Software Quality Model. Based on their feedback and the evaluation results, the researchers will make the necessary revisions and present the finalized platform back to the users. If the College of Computing Studies decides to officially adopt the system, the researchers will hand over the platform along with a complete user manual. This documentation will guide the instructors and administrators on how to manage the system properly. A formal letter of agreement will also be provided to state that the platform is willingly turned over to the department, and the researchers will no longer be required to maintain or update it after the study concludes.

Table 1: Implementation Plan

STRATEGY	ACTIVITIES	PERSON INVOLVED	DURATION
Approval from the College of Computing Studies	Letters of Request for System Deployment	Researchers, College Dean, and CCS Instructors	1 – 3 Days
System Setup and Server Deployment	Configuration of PostgreSQL, FastAPI, and Judge0 Sandbox	Researchers	1 - 2 Days
Information Distribution	Distribution of System User Manuals and Module Guides	Researchers and CCS Instructors	1 Day
System Orientation and Training	Hands-On Platform Navigation and IDE Usage Training	Researchers, CCS Instructors, and Third-Year CS Students	2 - 3 Days
ISO/IEC 25010 Software Evaluation	System Deployment for Agile Sampling and ISO/IEC 25010 Survey Data Collection	Researchers, CCS Instructors, and Third-Year CS Students	1 - 2 Weeks

A. **Server Requirements (Hosting, Database, and Code Execution)** These are the specifications required to host the system, run the Judge0 sandboxing environment, and manage the PostgreSQL database.

ITEM	REQUIREMENTS	
	Minimum	Recommended
Processor	4-Core CPU	8-Core CPU
Memory (RAM)	8 GB RAM	16 GB RAM
Storage Space	20 GB SSD	30 GB SSD
Operating System	Windows 10 64-bit	Windows 11 64-bit
Required Software	Docker, Python 3.10+, PostgreSQL	Docker, Python 3.10+, PostgreSQL
Network (For Server)	50 Mbps Stable Broadband Connection (or Cloud Hosting default bandwidth)	100 Mbps or higher Fiber Optic Connection (or Dedicated Cloud Bandwidth)

B. **Client Requirements (Students and CCS Instructors)** These are the specifications required for the end-users to access the platform and run the built-in code editor smoothly.

ITEM	REQUIREMENTS	
	Minimum	Recommended
Processor	2-Core CPU (Intel Core i3 / AMD Ryzen 3 or equivalent)	4-Core CPU (Intel Core i5 / AMD Ryzen 5 or equivalent)
Memory (RAM)	4 GB RAM	8 GB RAM
Operating System	Windows 10, macOS, or Linux	Windows 11, macOS, or Linux
Web Browser	Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari (Updated versions)	Google Chrome or Microsoft Edge (Latest versions for best Monaco Editor compatibility)
Internet Connection	2 Mbps Stable Connection	5 Mbps or higher Stable Connection

ISO/IEC 25010 SOFTWARE QUALITY MODEL

The ISO/IEC 25010 Software Quality Model is an international standard used by developers and quality assurance testers to assess the overall quality of a software product. It is a well-known model that originally has eight main quality characteristics. For this study, the researchers selected specific attributes that best fit the evaluation of the Web-Based Python Integrated Development Environment (IDE). The following are the attributes used by the researchers:

Functional Suitability is the ability of the web-based Python IDE to meet the specific coding, behavioral tracking, and structural feedback functions required to assess advanced programming logic and ensure academic integrity.

Performance Efficiency defines the capacity of the system to handle multiple students compiling complex codes simultaneously, manage system traffic, and process Abstract Syntax Tree (AST) data with fast response times.

Usability focuses on the creation of a user-friendly interface where the code editor, navigation, and real-time dashboard are easy for both third-year students and CCS instructors to utilize without unnecessary complexity.

Reliability and Security involve the measures the system takes to run advanced Python scripts safely in a sandboxed environment without crashing, and to protect student data and behavioral logs from unauthorized access.

RESPONDENTS OF THE STUDY

The study selected respondents based on their relevance to the research objectives and their capability to assess the technical integration of the system. The primary participants include third-year Computer Science students and instructors at the College of Computing Studies (CCS) of Pampanga State University (PAMSU). To conduct the software evaluation, the researchers will utilize agile sampling to iteratively select the respondents who will test the newly developed Python Web IDE during its deployment phases. The participating third-year students will execute various programming activities, including solo practice, laboratory assignments, and mock quizzes, to test the system's

post-submission AST structural feedback and behavioral tracking capabilities, while the CCS faculty members will evaluate the behavioral tracking logs and dashboard management. After the deployment and testing phase, both the students and the faculty members will answer a structured survey to assess the system's performance, functional suitability, usability, and overall technical quality strictly based on the ISO/IEC 25010 Software Quality Model.

Table 3: *Respondents of the Study*

Domain	No. of Respondents	Description
IT Experts and Instructors	5	IT professionals and instructors from the College of Computing Studies who will evaluate the system's technical quality, security, and performance based on the ISO/IEC 25010 model.
Third-Year Computer Science Students	30	Selected third-year BSCS students who will be divided equally into Control and Experimental groups to participate in the pre-test, utilize their designated coding environments, and answer the post-test and usability evaluations.
Total Respondents	35	

RESEARCH TOOLS AND INSTRUMENTS

The research instruments include a structured system evaluation survey and automated database log extractions. The objective is to collect accurate data by conducting a system deployment phase and a subsequent technical evaluation. The evaluation questionnaire for the technical aspect of the platform is formulated based on the ISO/IEC 25010 Software Quality Model.

The ISO/IEC 25010 framework offers an organized method for assessing and improving software quality to make the system reach its full potential. While the original model has eight product quality criteria, the tools employed in this study specifically focus on Functional Suitability, Performance Efficiency, Usability, and Reliability and Security. The specific instruments used in this study are outlined below:

1. System Evaluation Questionnaire

After the testing phase, a close-ended survey questionnaire based on the ISO/IEC 25010 model will be distributed to IT experts, CCS instructors, and the student respondents. Analyzing their responses will allow the researchers to determine the functional suitability, usability, and overall technical quality of the newly developed Web-Based Python IDE.

2. Automated System Logs

The researchers will utilize the built-in tracking features of the Python Web IDE as a secondary data gathering tool. This involves extracting the post-submission AST structural feedback results and the tab-switching

behavioral logs from the PostgreSQL database to objectively evaluate the system's performance in ensuring academic integrity during coding sessions.

Data Gathering Procedure

The primary method of data collection will be through database log extraction and online Google Forms. The following steps will be executed to collect the necessary data:

1. The researchers will obtain formal authorization from the Dean and administrators of the College of Computing Studies before conducting any system deployment and data collection.
2. The researchers will coordinate with the CCS instructors to deploy the Python Web IDE to the selected third-year BSCS students using agile sampling.
3. During the implementation phase, the participating students will utilize the newly developed Python Web IDE for different coding scenarios, such as standard laboratory assignments and supervised mock quizzes. This setup will thoroughly test the system's ability to handle concurrent post-submission AST feedback and accurately record behavioral tracking logs during high-pressure coding tasks.
4. Finally, the researchers will extract the automated logs from the database. Simultaneously, the ISO/IEC 25010 system evaluation questionnaire will be distributed to the IT experts, instructors, and student respondents to finalize the data gathering process.

Sampling Technique

The study will gather primary data through targeted coding scenarios and survey questionnaires. The researchers will utilize a non-probability sampling method, specifically the agile sampling technique, to iteratively select the respondents. Agile sampling was chosen because it allows the researchers to deliberately select individuals who are most relevant to the core objectives of the study and the software deployment setup.

For this research, the target participants are the third-year Computer Science students who already possess foundational coding knowledge and are suitable for executing advanced programming tasks, alongside CCS professionals and instructors at the College of Computing Studies. This sampling technique is highly advantageous for system development studies as it ensures that the feedback regarding the code editor, the post-submission AST structural feedback, and the system performance comes directly from the intended end-users and qualified technical experts. By focusing strictly on these specific groups during the testing phases, the researchers can obtain highly valid and reliable data for the ISO/IEC 25010 evaluation without the need to test the entire student population.

STATISTICAL TREATMENT OF THE DATA

To interpret the gathered data accurately, the researchers will utilize appropriate statistical tools. Since the study employs agile sampling with a targeted group of respondents, population-based sampling formulas are not

required. Instead, the researchers will exclusively use statistical methods designed to evaluate the software quality based on the system testing phases.

1. Weighted Mean

This statistical tool will be used to tally and calculate the responses from the Likert-scale questionnaires. It will be applied to interpret the system evaluation conducted by the IT experts, CCS instructors, and student respondents strictly based on the ISO/IEC 25010 Software Quality Model.

$$WM = (\Sigma fx) / N$$

where:

WM = Weighted Mean

Σfx = Sum of the products of the frequency and the corresponding weight

N = Total number of respondents

DATA ANALYSIS

The primary statistical tool that will be employed in the data analysis is the weighted mean. The weighted mean will be calculated to determine the average responses for each item in the ISO/IEC 25010 system evaluation questionnaire. Each item in the evaluation offers five response options: 5 (Strongly Agree), 4 (Agree), 3 (Neutral), 2 (Disagree), and 1 (Strongly Disagree). To accurately interpret the computed weighted mean, the following specific score ranges and their corresponding descriptive interpretations will be used:

Table 4. *Likert Scale***LIKERT SCALE**

Point	Scale Range	Interpretation (System Quality)
5	4.20 – 5.00	Strongly Agree (Highly Acceptable)
4	3.40 – 4.19	Agree (Acceptable)
3	2.60 – 3.39	Neutral (Moderately Acceptable)
2	1.80 – 2.59	Disagree (Slightly Acceptable)
1	1.00 – 1.79	Strongly Disagree (Not Acceptable)

Table 5. *Overall Evaluation Rating Scale and Interpretation*

Rating	Descriptive Equivalent	Interpretation
5	Strongly Agree (Highly Acceptable)	The system operated remarkably, fulfilling or surpassing all expectations. The secure code execution, post-submission AST structural feedback, and behavioral tracking features were highly effective.
4	Agree (Acceptable)	The system functioned effectively, meeting most expectations. Users generally had a favorable experience navigating the platform, although there are minor areas for improvement in the interface or feedback delivery.
3	Neutral (Moderately Acceptable)	The system's performance is acceptable but exhibits noticeable limitations. Users experienced a functional IDE, but specific evaluation features or navigation interfaces require refinement to be fully effective.
2	Disagree (Slightly Acceptable)	The system experienced several issues and shortcomings, resulting in a mixed user experience. Significant improvements are necessary for the platform's stability, security, or AST structural feedback delivery.
1	Strongly Disagree (Not Acceptable)	The system functioned poorly, encountering numerous issues, crashes, or execution limitations. Users experienced frustration and severe dissatisfaction with the platform's overall performance.

This table presents a 5-point Likert scale, enabling respondents to rate the platform from "Strongly Agree" to "Strongly Disagree." This detailed breakdown facilitates a clear, descriptive evaluation of the users' overall experience and the technical capability of the system based on the ISO/IEC 25010 standards.

Table 6: *Verbal Interpretation of Evaluator's Responses Regarding the Functional Suitability of the System*

Range	Interpretation	Verbal Interpretation
4.20 – 5.00	Strongly Agree	Very Functional
3.40 – 4.19	Agree	Functional
2.60 – 3.39	Neutral	Moderately Functional
1.80 – 2.59	Disagree	Less Functional
1.00 – 1.79	Strongly Disagree	Non-Functional

Table 7: *Verbal Interpretation of Evaluator's Responses Regarding the Performance Efficiency of the System*

Range	Interpretation	Verbal Interpretation
4.20 - 5.00	Strongly Agree	Very Efficient
3.40 - 4.19	Agree	Efficient
2.60 - 3.39	Neutral	Moderately Efficient
1.80 - 2.59	Disagree	Less Efficient
1.00 - 1.79	Strongly Disagree	Inefficient

Table 8: *Verbal Interpretation of Evaluator's Responses Regarding the Usability of the System*

Range	Interpretation	Verbal Interpretation
4.20 - 5.00	Strongly Agree	Very Usable
3.40 - 4.19	Agree	Usable
2.60 - 3.39	Neutral	Moderately Usable
1.80 - 2.59	Disagree	Less Usable
1.00 - 1.79	Strongly Disagree	Not Usable

Table 9: *Verbal Interpretation of Evaluator's Responses Regarding the Reliability and Security of the System*

Range	Interpretation	Verbal Interpretation
4.20 - 5.00	Strongly Agree	Very Reliable and Secure
3.40 - 4.19	Agree	Reliable and Secure
2.60 - 3.39	Neutral	Moderately Reliable and Secure
1.80 - 2.59	Disagree	Less Reliable and Secure
1.00 - 1.79	Strongly Disagree	Not Reliable and Secure

REFERENCES

Adegbite, A. A., & Kotzé, E. (2025). Detection of AI-generated code in programming education. *African Journal of Information and Communication*.
<https://journals.co.za/doi/10.23962/ajic.i35.21309>

Alifya, S. N. F. H., Dewi, S. S., & Anandari, S. D. R. (2025). Evaluating EduTrack system quality using ISO/IEC 25010 and the DeLone and McLean model. *Journal of Embedded System Security and Intelligent Systems*, 6(3), 427–440. <https://doi.org/10.59562/jessi.v6i3.10266>

Alkinani, E. A. (2026). Multimodal transformer framework for real-time cheating detection in online assessments and e-learning platforms. *SN Computer Science*, 7(1). <https://doi.org/10.1007/s42979-025-04718-3>

Bato, B. E., & Pomperada, J. R. (2025). Automated grading system with student performance analytics. *Technium Romanian Journal of Applied Sciences and Technology*, 30, 58–75. <https://doi.org/10.47577/technium.v30i.12871>

Centeno, C. J., Mababa, J., Tano, I. M., Piad, K., Lagman, A., Espino, J., & Victoriano, J. (2025). Academic study plan recommender and simulator system using forward chaining and heuristic algorithms. *International Journal of Latest Technology in Engineering Management & Applied Science*, 14(1), 201–208. <https://doi.org/10.51583/ijltemas.2025.1401021>

Chen, J., & Wang, L. (2025). Architecture of online code execution systems using sandboxing and distributed queues. *Journal of Cloud Computing and Educational Technology*. <https://doi.org/10.3390/app151910403>

Chen, R., Jiang, S., Shen, J., Moon, A., & Wei, L. (2025). Examining the usage of generative AI models in student learning activities for software programming. *arXiv*. <https://doi.org/10.48550/arXiv.2511.13271>

Elnaffar, S., Rashidi, F., & Abualkishik, A. (2025). Teaching with AI: A systematic review of chatbots and generative tools in programming education. *arXiv*. <https://doi.org/10.48550/arXiv.2510.03884>

Frontiers. (2025). Using pseudo-AI submissions for detecting AI-generated code. *Frontiers in Computer Science*. <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1549761/full>

Gandhi, N., Gopalan, K., & Prasad, P. (2024). A support vector machine based approach for plagiarism detection in Python code submissions in undergraduate settings. *Frontiers in Computer Science*, 6, 1393723. <https://doi.org/10.3389/fcomp.2024.1393723>

Gangothri, C. L., & Chandrappa, S. (2026). An integrated plagiarism detection technique combining natural language processing (NLP) methodologies with statistical similarity metrics. *Zenodo*. <https://doi.org/10.5281/zenodo.19554540>

Gao, Z., Yan, H., Liu, J., Zhang, X., Lin, Y., & Zhang, Y. (2025). Tracing distinct learning trajectories in introductory programming using code metrics and

engagement data. *International Journal of STEM Education*, 12, Article 46.

<https://doi.org/10.1186/s40594-025-00546-2>

Garcia, M. B. (2025). Teaching and learning computer programming using ChatGPT: A rapid review of literature amid the rise of generative AI technologies.

Education and Information Technologies.

<https://doi.org/10.1007/s10639-025-13452-5>

International Journal of Engineering Research & Technology. (2026). Learn to code fast: Cloud-based sandboxing and real-time evaluation.

<https://doi.org/10.5281/zenodo.18206824>

Lepp, M., et al. (2025). Does generative AI help in learning programming? Evidence from an object-oriented programming course. *Computers and Education: Artificial Intelligence*. <https://doi.org/10.1016/j.chbr.2025.100642>

Li, S., Liu, J., & Dong, Q. (2025). Generative artificial intelligence-supported programming education: Effects on learning performance, self-efficacy and processes. *Australasian Journal of Educational Technology*, 41(3), 1-25. <https://doi.org/10.14742/ajet.9932>

Martinez-Gil, J. (2024). Source code clone detection using unsupervised similarity measures. *arXiv*. <https://arxiv.org/html/2401.09885v1>

Nasr, N. R., et al. (2025). Impact of ChatGPT on critical thinking. *Education Sciences*, 15(9), 1198. <https://www.mdpi.com/2227-7102/15/9/1198>

Nathaniel, J. (2025). AI in the classroom: Exploring students' interaction with ChatGPT in programming learning. *Education and Information Technologies*. <https://doi.org/10.1007/s10639-025-13337-7>

Nurhasan, U., Prasetya, D. D., & Patmanthara, S. (2025). Abstract syntax tree model for minimizing false negative in semantic evaluation of Python fill-in-the-blank. *Journal of Applied Informatics and Computing*. <https://jurnal.polibatam.ac.id/index.php/JAIC/article/view/11090>

Oliveira, C., Silva, L., & Brunet, J. (2025). Beyond functionality: Automating algorithm design evaluation in introductory programming courses. *SciTePress*. <https://www.scitepress.org/Papers/2025/132420/132420.pdf>

Özgül, E. (2026). A systematic review of generative AI in programming education. *Kastamonu Education Journal (KEFAD)*. <https://dergipark.org.tr/en/pub/kefdergi/article/1878122>

PacificCert Quality Assurance. (2025). ISO/IEC 25010:2023 guide to software product quality standards. *PacificCert Global Standards*. <https://blog.pacificcert.com/iso-25010-2025-guide/>

Park, H., Kim, Y., Lee, K., Jin, S., Kim, J., Heo, Y., Kim, G., & Kim, E. (2025). CodeDive: A web-based IDE with real-time code activity monitoring for programming education. *Applied Sciences*, 15(19), 10403. <https://doi.org/10.3390/app151910403>

Parvathy, R., Thushara, M. G., & Kannimoola, J. M. (2025). Automated code assessment and feedback: A comprehensive model for improved programming education. *IEEE Access*, *13*, 56642–56658. <https://doi.org/10.1109/ACCESS.2025.3554838>

Ramizares, M., & Ramizares, M. (2025). Development of a web-based adaptive multi-event tabulation system. *International Journal for Multidisciplinary Research*, *7*(4). <https://doi.org/10.36948/ijfmr.2025.v07i04.54635>

Real Python Research Group. (2026). Asynchronous tasks, with Django and Celery: Maintenance of app responsiveness. *Real Python Academic Series*. <https://realpython.com/asynchronous-tasks-2026/>

Senjaya, W. F., Nathasya, R. A., Liliawati, S. L., Karnalim, O., Vivaldy, T., & Arsito, J. L. (2025). Outsmarting disguises: Evaluating SSTRANGE for plagiarism detection in computer graphics code. *2025 13th International Conference on Orange Technology (ICOT)*. <https://doi.org/10.1109/ICOT68409.2025.11425138>

Socrates, T. P., Fahlevi, R., Ikram, R., & Wahyudhie, M. I. (2022). Smart detector: Anti-cheating exam detection based on IoT and biosecurity management. *SPECTRUM*, *1*(02). <https://doi.org/10.54482/spectrum.v1i02.169>

Soppari, K., Gunturi, V. S. R., Raparathi, H., & Bimagoni, M. G. (2025). A study on Codeloop Django-based coding platform. *World Journal of Advanced Research and Reviews*, 26(3), 2586–2591. <https://doi.org/10.30574/wjarr.2025.26.3.2455>

Stoyanova, D., Stoyanova-Petrova, S., Shotarova, S., Lyubomirov, S., & Mileva, N. (2026). ChatGPT in programming education: An empirical study on its impact. *Education Sciences*, 16(1), 19. <https://doi.org/10.3390/educsci16010019>

Suryani, A., & Pratama, R. (2025). Evaluating EduTrack system quality using ISO/IEC 25010 and the DeLone and McLean model. *Journal of Education and Software Systems Innovation (JESSI)*, 12(1), 45–58. <https://journal.unm.ac.id/index.php/JESSI/article/view/10266>

Wu, X., Zhu, P., Zhang, J., et al. (2026). ChatGPT's impact on student learning outcomes: A meta-analysis of 35 experimental studies. *Humanities and Social Sciences Communications*. <https://doi.org/10.1057/s41599-026-07019-z>

Wulandari, B., Rahman, D. F., Fitriyanto, D., Dewanto, S. A., & Munir, M. (2026). Development and ISO 25010 evaluation of a web-based laboratory system. *Jurnal Ilmiah Ilmu Terapan Universitas Jambi*, 10(1), 1–16. <https://doi.org/10.22437/jiituj.v10i1.47430>

Zhao, G., et al. (2025). AI-assisted programming learning and computational thinking: Effects and implications. *Journal of Educational Computing Research*. <https://doi.org/10.1177/07356331251336154>